



SCS 2303

SOFTWARE ARCHITECTURES

UNIVERSITY COURSE
ENROLLMENT SYSTEM

ASSIGNMENT 03

Group Members

- 24000507 - D. M. M. A. Dissanayake
- 24000035 - T. D. Abeydeera
- 24000094 - A. A. M. B. Amarasinghe
- 24000282 - S. J. B. Boyagoda
- 24001937 - M. H. M. Shafkath
- 24002224 - W. M. D. S. Wickramasinghe

Table of Contents

1. Architectural Design	4
1.1. Problems with the Existing Application	4
1.2. Microservices as the Proposed Architecture	5
1.3 Other Architectural Patterns Considered	6
1.3.1 Service Oriented Architecture	6
1.3.2 3-Tier Architecture	6
1.4 Architectural Diagram	7
1.5. System Services and Components	8
1.5.1. API Gateway	8
1.5.2. Identity Service	8
1.5.3. Enrollment Service	8
1.5.4. Faculty Service	8
1.5.5. Administrator Service	9
1.5.6. Notification Service	9
1.5.7. Service Communication	9
2. Application of Design Patterns	10
2.1. Factory Method Pattern	10
2.2. Chain of Responsibility Pattern	12
2.3. Observer Pattern	14
2.4 Command Pattern	15
2.5 State Pattern	17
3. Design Diagrams	19
3.1. Sequence Diagrams	19
3.2. Activity Diagrams	32
3.3. State Diagrams	44
4. Software Engineering Principles Demonstrated	47
4.1. Encapsulation	47
4.2. Programming to an Interface	48
4.3. Composition Over Inheritance	49
4.4. SOLID Principles	49
4.4.1. Single Responsibility Principle	49
4.4.2. Open/Closed Principle	50
4.4.3. Liskov Substitution Principle	51
4.4.4. Interface Segregation Principle	52
4.4.5. Dependency Inversion Principle	54
5. Conclusion	56

1. Architectural Design

1.1. Problems with the Existing Application

Nexus University's existing course enrollment system, "LegacyEnroll", has been in operation for more than 20 years. As the university's student population has grown, the system has become increasingly difficult to maintain and slow to operate. It also lacks modern capabilities such as real-time updates, mobile accessibility, and a user-friendly interface. These limitations have created the need for a more flexible, scalable, and maintainable replacement.

Maintainability

LegacyEnroll contains the functionality required by different university roles within a single application. Student enrollment, faculty management, administration, reporting, and notifications are therefore part of the same overall codebase.

As the system has grown, this has made the application increasingly difficult to understand and modify. Changes made to one area can affect other areas of the system, increasing the effort required for development and testing. The size and age of the application also make it more difficult to identify the source of problems and safely introduce changes.

Scalability

The university expects the new system to support a rapidly growing student population and a high volume of simultaneous users, particularly during peak enrollment periods.

In the monolithic system, all major functionality forms part of the same application. As a result, increased demand in one area also places pressure on the deployment as a whole. A large increase in enrollment requests during registration affects the same application that is responsible for administration, reporting, and other functionality.

This makes it difficult to allocate resources according to the actual workload of individual parts of the system.

Limited Accessibility

LegacyEnroll lacks modern access through mobile applications and does not provide the level of user experience expected from a contemporary university system.

The existing system was developed as a long-running application rather than as a backend designed to serve multiple types of client applications. This limits its ability to support different access platforms in a consistent manner.

Coupling of Notifications with Core Operations

In the existing monolithic structure, notification behaviour forms part of the same application as the core enrollment functionality. As these responsibilities are contained within one system, changes to notification functionality can introduce dependencies into the enrollment process.

Limited Fault Isolation

The system contains several distinct functional areas with different purposes and workloads. However, because these functions are part of the same monolithic application, a serious problem affecting the application can potentially affect multiple functions simultaneously.

Difficulty with Future Expansion

The university expects the new platform to remain flexible enough to integrate with future systems, including potential services such as a financial aid system.

In a long-established monolithic application, adding substantial new functionality can require changes across an increasingly large and interconnected codebase. As the number of functions grows, maintaining clear boundaries between existing and new functionality becomes progressively more difficult.

1.2. Microservices as the Proposed Architecture

Microservices Architecture was selected for NexusEnroll because the system contains several distinct business capabilities that can operate as independently deployable services. The proposed architecture consists of an Identity Service, Enrollment Service, Faculty Service, Administrator Service, Notification Service, and an API Gateway.

Maintainability Through Service Separation

Each service has a defined responsibility and limited scope. This reduces coupling between functional areas and allows changes to be concentrated within the service responsible for them.

The Enrollment Service, for instance, manages courses and enrollments without containing the implementation of faculty management or notification delivery.

Independent Scalability

Services can be scaled according to their individual workloads. The Enrollment Service can therefore handle increased demand during registration without requiring the entire system to be scaled simultaneously.

Support for Multiple Client Platforms

The API Gateway provides a common entry point for external clients and routes requests to the appropriate service. The same backend APIs can therefore support both the university's web application and a future mobile application.

Decoupled Notifications

Notification processing is handled by a separate Notification Service. The Enrollment Service publishes relevant events without implementing notification delivery itself, allowing notification processing to remain independent of the enrollment workflow.

The Observer Pattern supports this decoupling within the implementation.

Fault Isolation

Because services run independently, failures can be contained to the affected functional area. A failure in reporting or administration does not inherently prevent the Enrollment Service from continuing to process student requests.

Future Extensibility

The service boundaries provide defined integration points for future functionality, including potential systems such as financial aid. New capabilities can therefore be introduced without restructuring the entire application.

1.3 Other Architectural Patterns Considered

1.3.1 Service Oriented Architecture

Service-Oriented Architecture was considered because it also structures functionality as communicating services. However, NexusEnroll benefits more from independently deployable and independently scalable services with clearly bounded responsibilities.

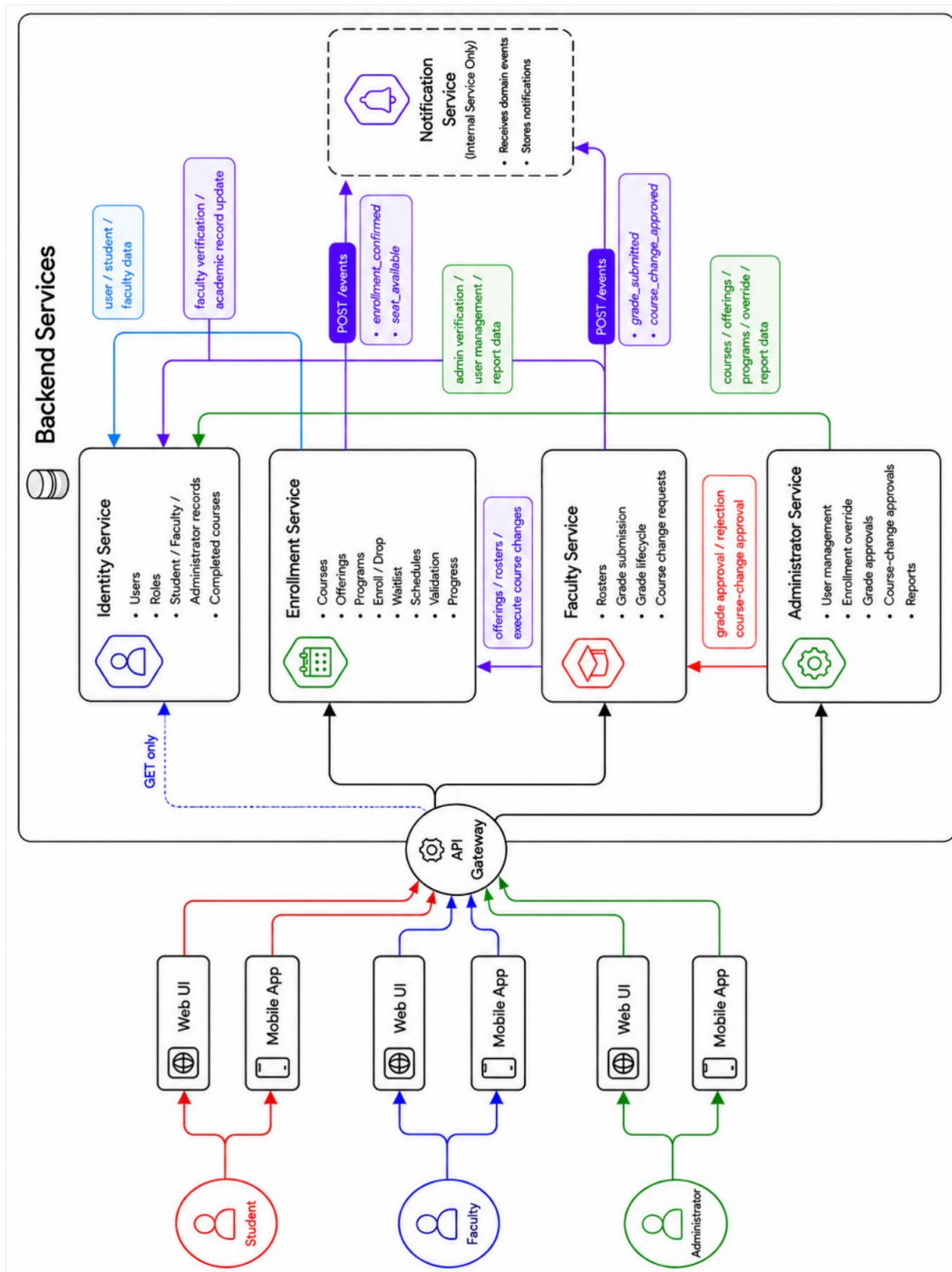
Microservices places stronger emphasis on these characteristics and therefore provides a closer fit to the required operating model. SOA remains a viable alternative, but was not selected as the primary architecture.

1.3.2 3-Tier Architecture

3-Tier Architecture separates presentation, business logic, and data access, providing useful internal structure. However, its tiers do not provide the same service-level independence required by NexusEnroll.

The university needs individual capabilities such as enrollment and notifications to be independently deployed, scaled, and isolated. Microservices provides this separation directly, while 3-tier principles can still be applied internally within individual services where appropriate.

1.4 Architectural Diagram



1.5. System Services and Components

NexusEnroll is composed of five microservices and an API Gateway. Each service owns a distinct area of the system and contains the business logic relevant to that responsibility. The services communicate through REST APIs, while event-based communication is used for notifications.

1.5.1. API Gateway

The API Gateway provides the main entry point for external clients and routes requests to the appropriate service. It keeps service-specific endpoints behind a single client-facing interface while allowing the individual services to remain independently deployable. The gateway is intentionally lightweight and focuses on routing rather than containing business logic.

1.5.2. Identity Service

The Identity Service manages accounts for students, faculty members, and administrators. It handles account creation, modification, deactivation, and retrieval of academic identity information such as completed courses. Account creation uses the **Factory Method Pattern**, with dedicated creators for each user type, allowing new account types to be introduced without placing creation logic directly in the API layer.

1.5.3. Enrollment Service

The Enrollment Service contains the core academic enrollment logic. It manages courses, course offerings, programmes, enrollments, waitlists, schedules, and academic progress, while also enforcing enrollment rules such as prerequisites, capacity, and timetable conflicts. These validation rules are implemented using the **Chain of Responsibility Pattern**, allowing each rule to be evaluated independently. The service also uses the **Observer Pattern** to publish events such as confirmed enrollments and available places without coupling notification delivery to the enrollment workflow.

1.5.4. Faculty Service

The Faculty Service manages functionality used by instructors, including class rosters, grade submissions, and course-change requests. Grade submissions use the **State Pattern** to control transitions between states such as Draft, Pending, Rejected, and Submitted. Course modifications use the **Command Pattern**, encapsulating individual changes such as modifying descriptions, adding prerequisites, or changing capacity as executable command objects.

1.5.5. Administrator Service

The Administrator Service provides higher-level administrative operations, including user management, course and programme management, enrollment overrides, approval of grades and course changes, and administrative reporting. Rather than duplicating the underlying business data, it coordinates operations across the Identity, Enrollment, and Faculty Services through their APIs. This allows administrative workflows to span multiple services while keeping ownership of the underlying logic within those services.

1.5.6. Notification Service

The Notification Service handles notifications generated by other parts of the system. It receives events such as course availability, grade submission, and approved course changes and converts them into user notifications. The service is intentionally separated from the operations that generate these events, with the **Observer Pattern** providing the connection between event-producing business logic and notification delivery.

1.5.7. Service Communication

The services communicate through REST APIs when one service requires data or an operation owned by another service. For event-driven operations, the producing service publishes a domain event which is handled by an observer that forwards it to the Notification Service. This combination allows request-response communication to be used where direct information is required while keeping notification processing independent from the core business operations.

2. Application of Design Patterns

The NexusEnroll implementation applies five object-oriented design patterns to address specific problems within the business logic. The following sections describe the purpose of each pattern, the problem it addresses, and its application in the implemented services.

2.1. Factory Method Pattern

Pattern overview

The Factory Method Pattern separates object creation from the code that uses the objects. Instead of directly instantiating different concrete classes, creation is delegated to appropriate creator objects. This reduces coupling between the code requesting an object and the concrete class being created.

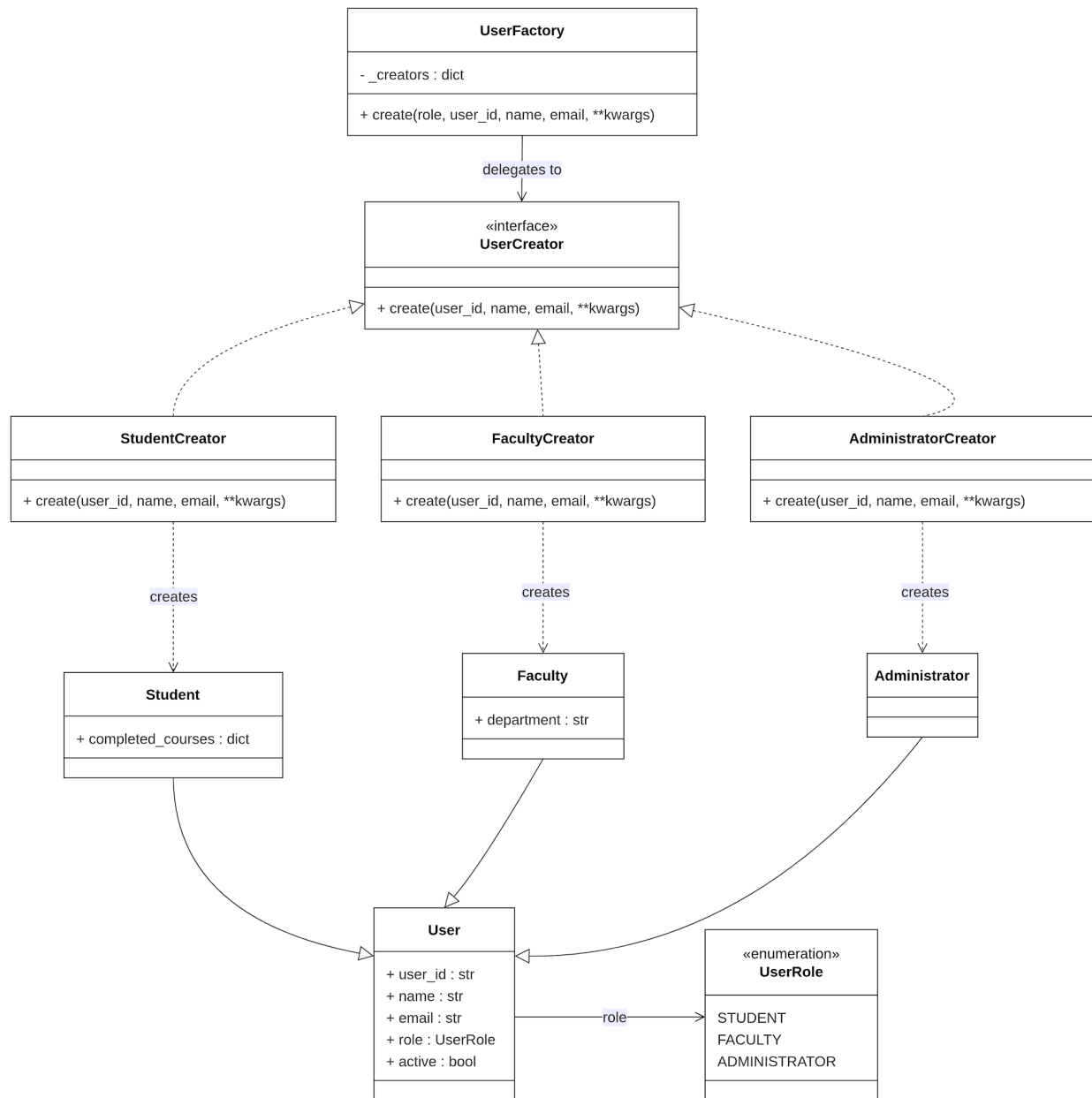
Problem addressed

The Identity Service needs to create different user types, including students, faculty members, and administrators. Without the pattern, account-creation logic would need to contain conditional logic that directly instantiates each user type. Adding another user type would then require modifying this existing creation logic.

Application in NexusEnroll

The `UserFactory` maintains a mapping between `UserRole` values and corresponding `UserCreator` implementations. `StudentCreator`, `FacultyCreator`, and `AdministratorCreator` each implement the common `UserCreator` interface and create their respective user types. The Identity Service therefore requests user creation through the factory without depending directly on the concrete creation process.

Class Diagram



The diagram shows the `UserCreator` abstraction, its concrete creator classes, the `UserFactory`, and the `User` subclasses they produce.

2.2. Chain of Responsibility Pattern

Pattern overview

The Chain of Responsibility Pattern passes a request through a sequence of handlers, with each handler responsible for a particular check or operation. A handler can either stop the process by rejecting the request or pass it to the next handler.

Problem addressed

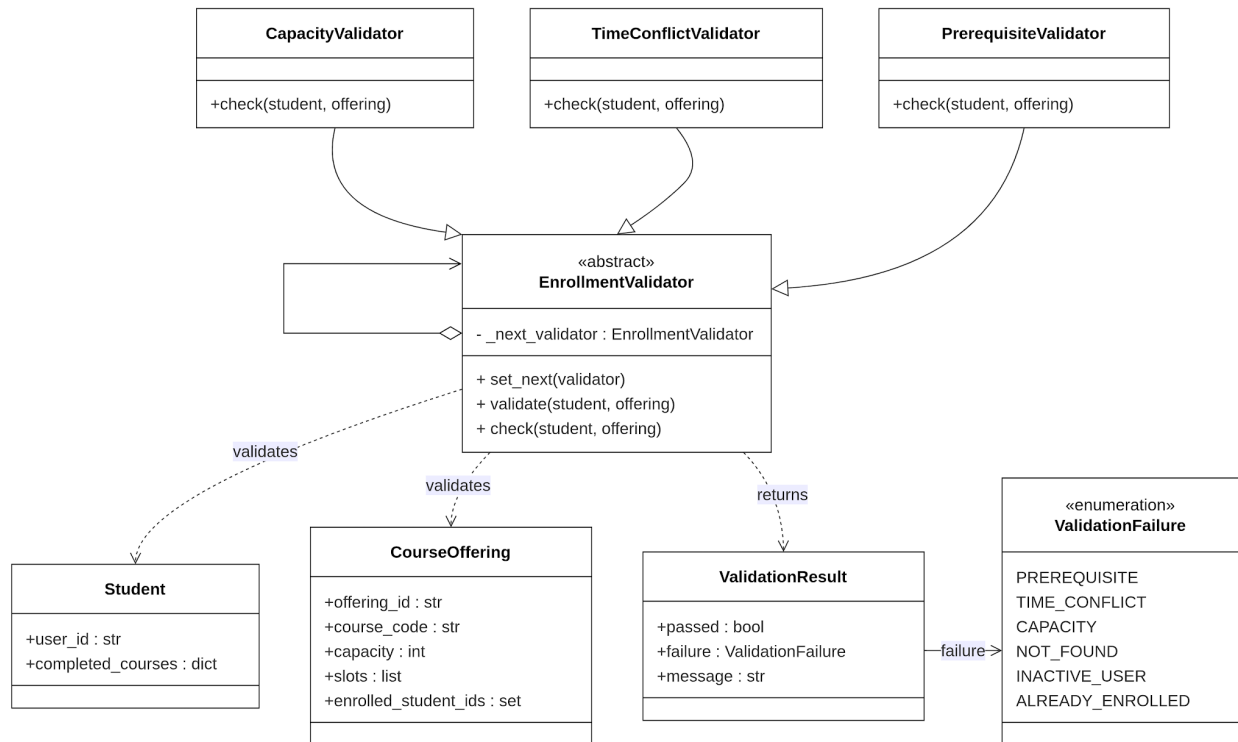
Course enrollment requires several independent validations: prerequisite requirements, timetable conflicts, and course capacity. Implementing these checks as one large conditional block would make the enrolment logic harder to read and harder to extend when additional validation rules are introduced.

Application in NexusEnroll

The `EnrollmentService` constructs a validation chain consisting of `PrerequisiteValidator`, `TimeConflictValidator`, and `CapacityValidator`. Each validator implements `EnrollmentValidator` and returns a `ValidationResult`. A failed validation stops the chain, while a successful validation passes the request to the next handler.

This allows individual validation rules to be modified or additional validators to be introduced without restructuring the entire enrolment process.

Class structure



The diagram shows **EnrollmentValidator**, the three concrete validators, **ValidationResult**, and the relationships that form the validation chain.

2.3. Observer Pattern

Pattern overview

The Observer Pattern allows an object to notify a set of dependent objects when an event occurs, without the object producing the event needing to know how each observer handles it.

Problem addressed

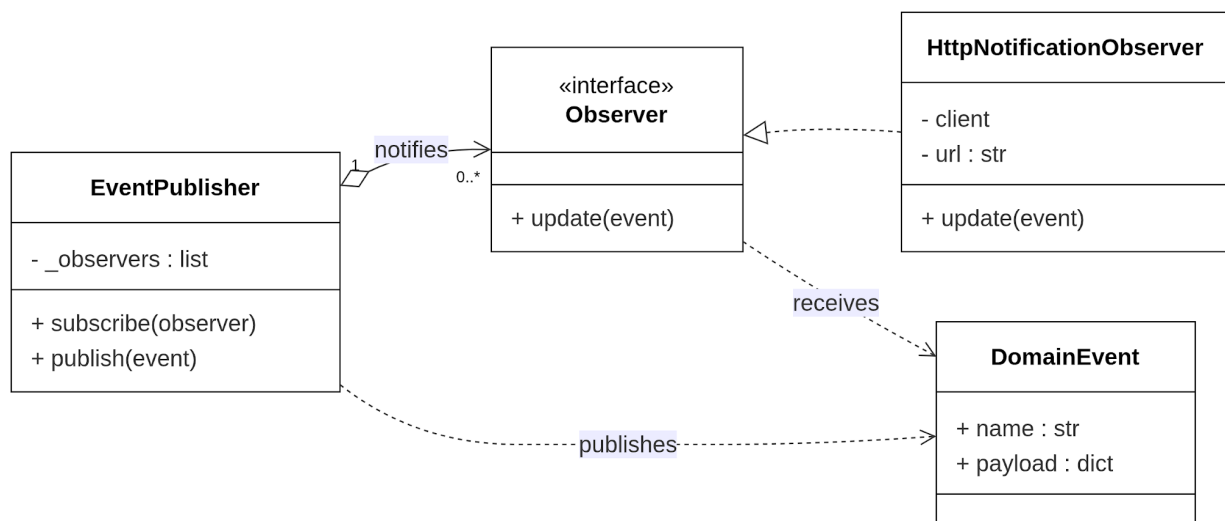
Core enrolment operations should not be responsible for notification delivery. Directly embedding notification logic into enrolment operations would couple the two concerns and could allow a notification failure to interfere with the underlying business transaction.

Application in NexusEnroll

The Enrollment Service uses an `EventPublisher` as the event source. Events are represented by the `DomainEvent` class and delivered to subscribed implementations of `Observer`.

`HttpNotificationObserver` is the concrete observer used by the application. It forwards domain events to the Notification Service through its API. The publisher isolates observer failures so that a notification failure does not roll back the underlying business operation. This is used for events such as confirmed enrolments and available places following a course drop.

Class structure



The diagram shows `EventPublisher`, `Observer`, `HttpNotificationObserver`, and `DomainEvent`, with the Notification Service represented as the external service receiving the published events.

2.4 Command Pattern

Pattern overview

The Command Pattern encapsulates an operation as an object. This separates the request for an operation from the code that performs it and allows different operations to be handled through a common interface.

Problem addressed

Faculty members can request several different types of course changes, such as modifying a description, adding a prerequisite, or changing capacity. Implementing all of these operations directly in the course-change workflow would require the workflow to contain separate logic for each modification.

Application in NexusEnroll

`CourseChangeCommand` defines the common `execute()` operation. The Faculty Service can work with any concrete command without needing to know its implementation.

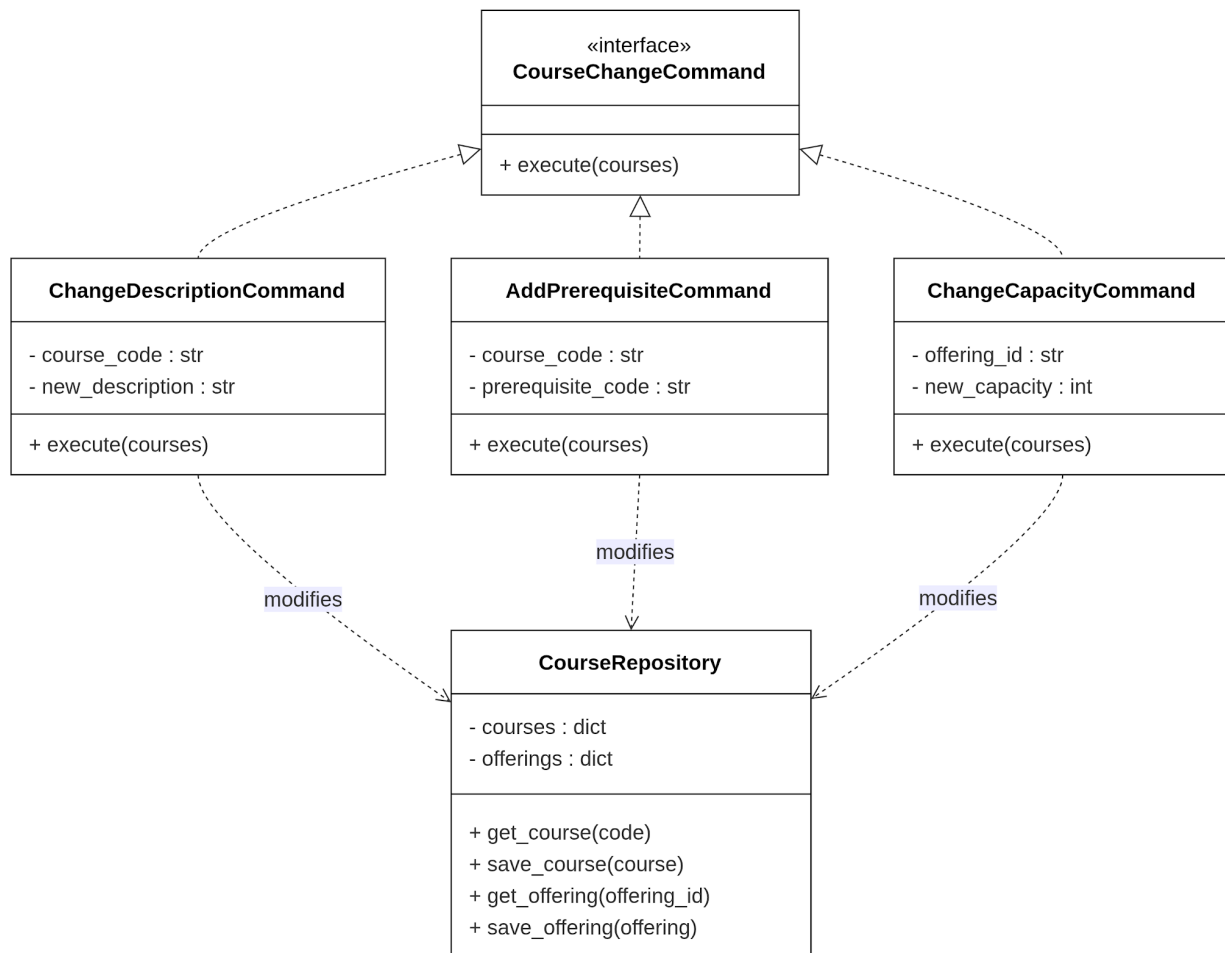
The current implementation provides three concrete commands:

- `ChangeDescriptionCommand`
- `AddPrerequisiteCommand`
- `ChangeCapacityCommand`

The `command_from_dto()` function selects and constructs the appropriate command from the requested course-change action. The resulting command then applies the modification through the course repositories.

This structure also fits the approval workflow: a pending course-change request can retain the operation as an executable command until an administrator approves it.

Class structure



2.5 State Pattern

Pattern overview

The State Pattern allows an object to alter its behaviour according to its current state. Instead of placing all state-dependent behaviour into conditional statements, each state is represented by a separate object.

Problem addressed

Grade submissions have a defined lifecycle. A grade can be in states such as Draft, Pending, Rejected, or Submitted, and the operations allowed at each stage are different. Without the State Pattern, these rules would need to be implemented through repeated conditionals based on the current grade status.

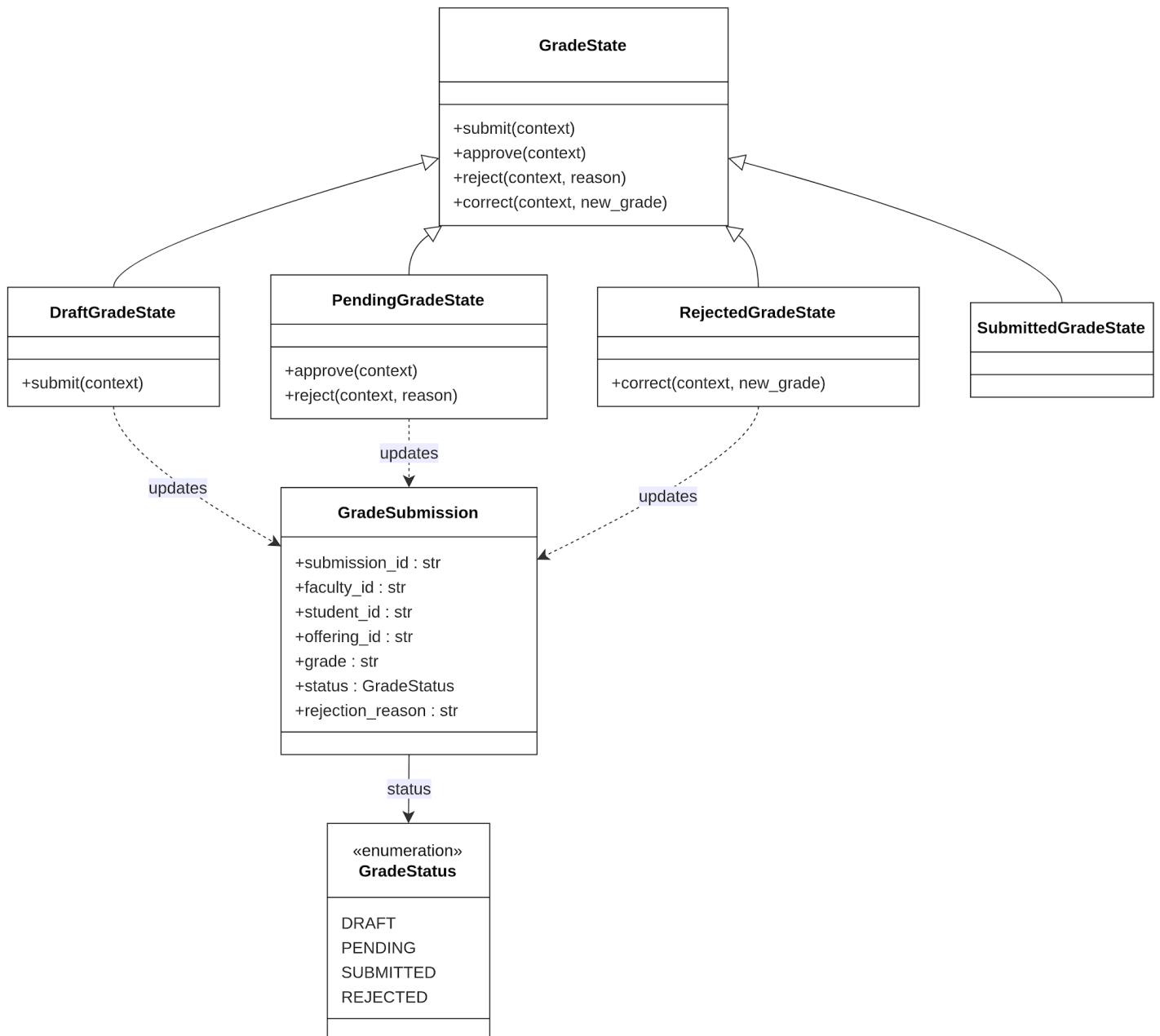
Application in NexusEnroll

`GradeSubmission` uses a corresponding `GradeState` implementation to determine which operations are valid. The current implementation defines `DraftGradeState`, `PendingGradeState`, `RejectedGradeState`, and `SubmittedGradeState`.

For example, a draft grade can be submitted, a pending grade can be approved or rejected, and a rejected grade can be corrected and returned to the Draft state. Unsupported operations raise an error rather than silently changing the grade.

The `state_for()` function maps the current `GradeStatus` to its corresponding state object.

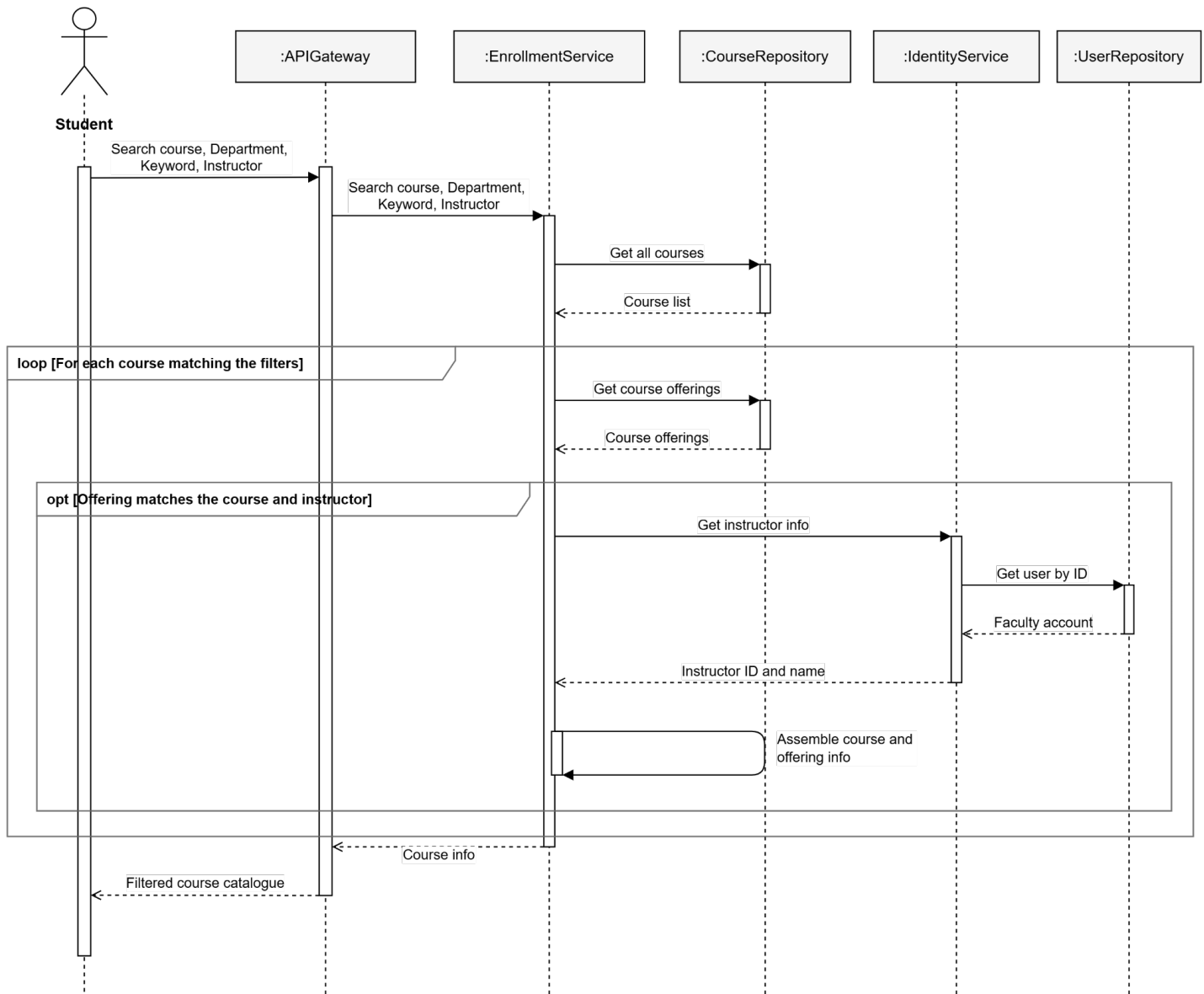
Class structure



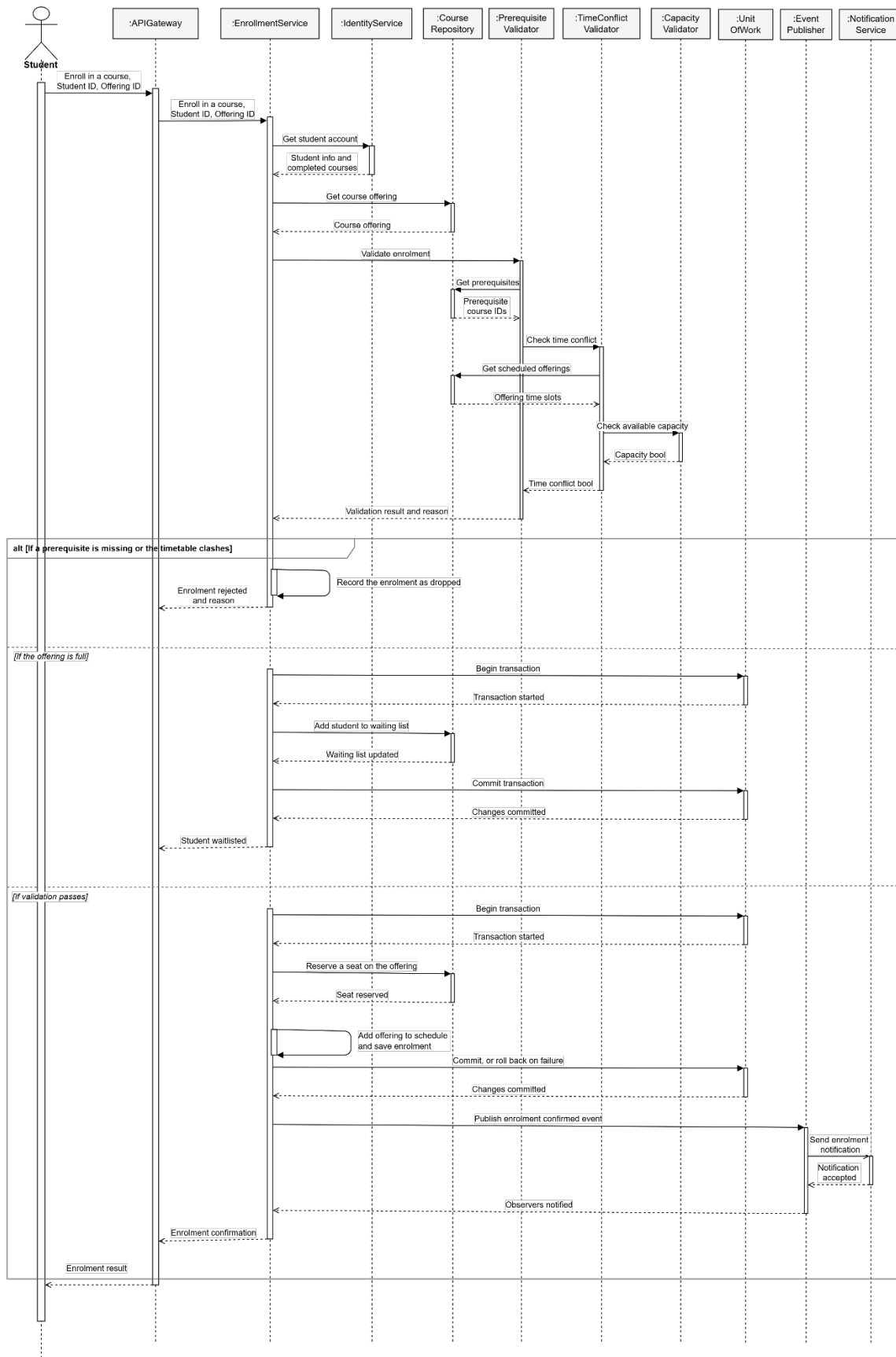
3. Design Diagrams

3.1. Sequence Diagrams

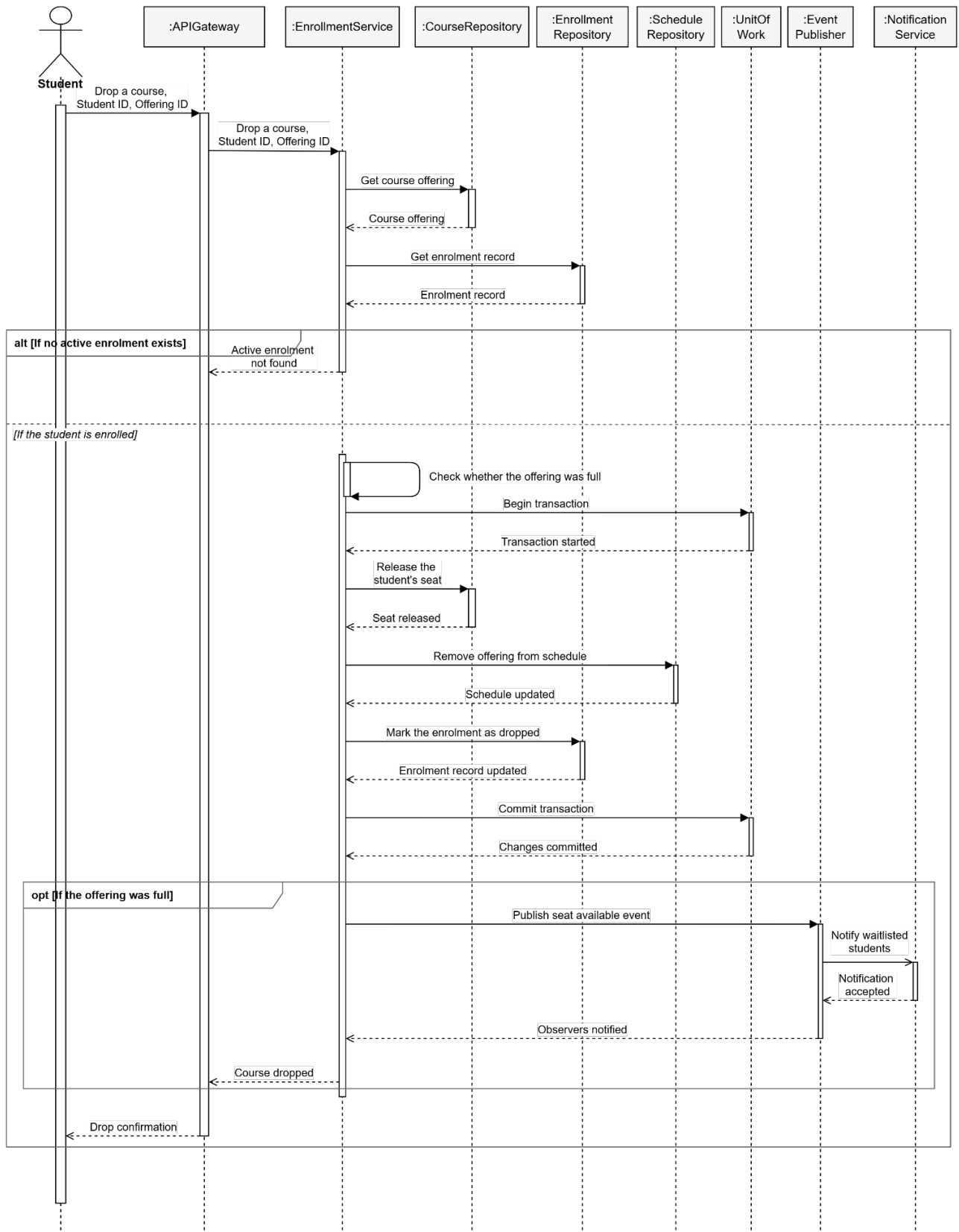
Sequence Diagram 1 - Browse / Search Course Catalogue (Student)



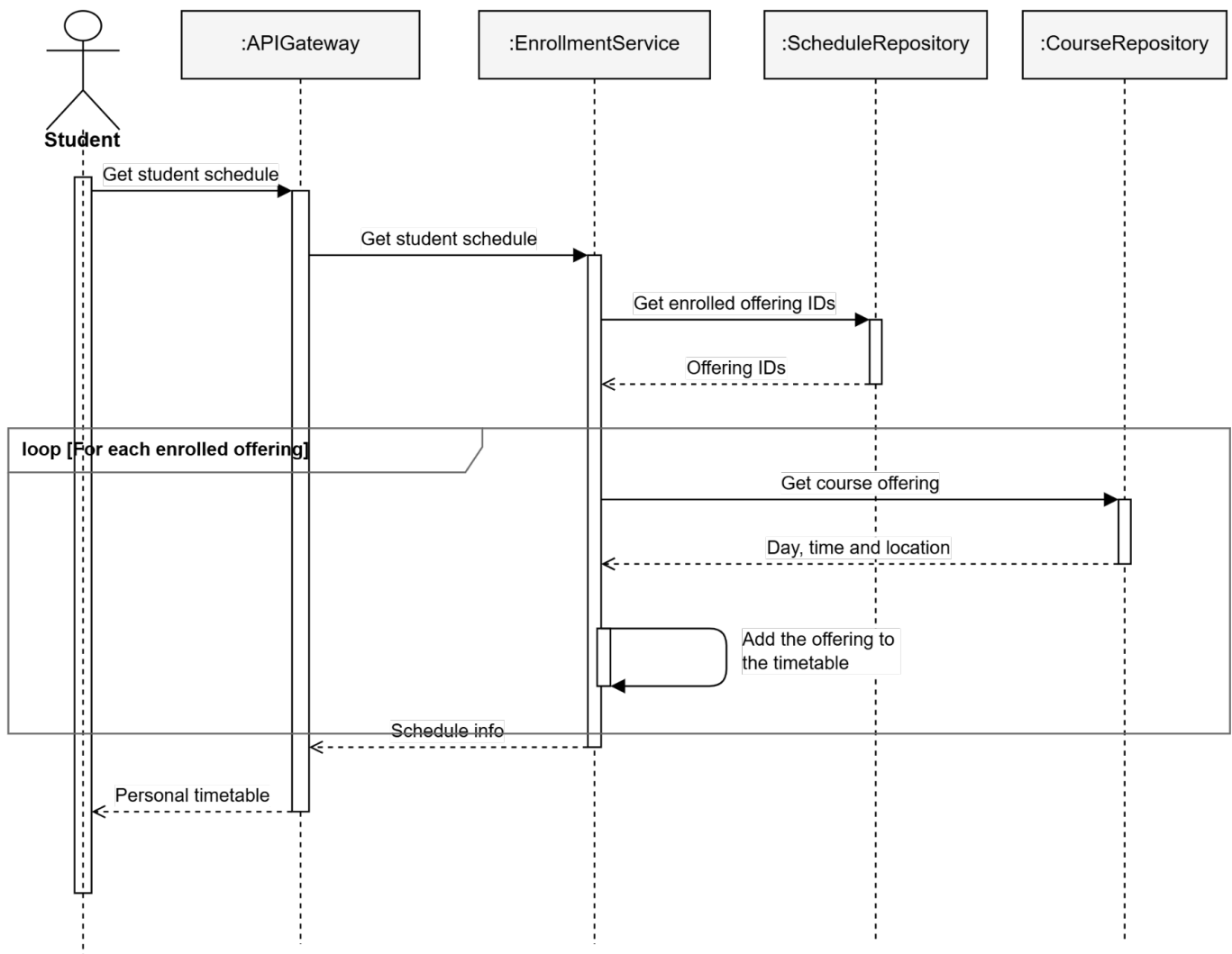
Sequence Diagram 2 - Course Registration and Enrolment (Student)



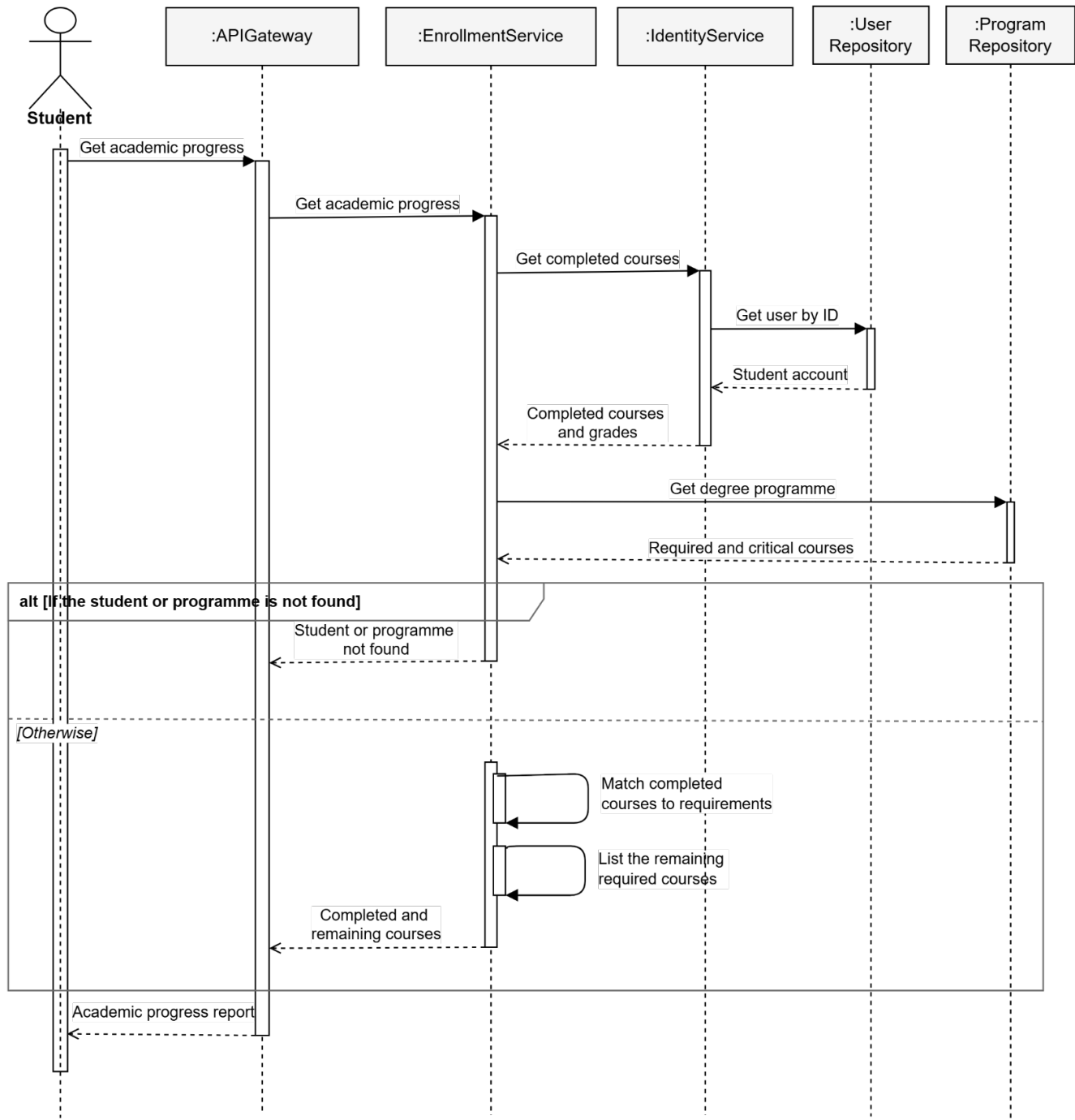
Sequence Diagram 3 - Drop a Course (Student)



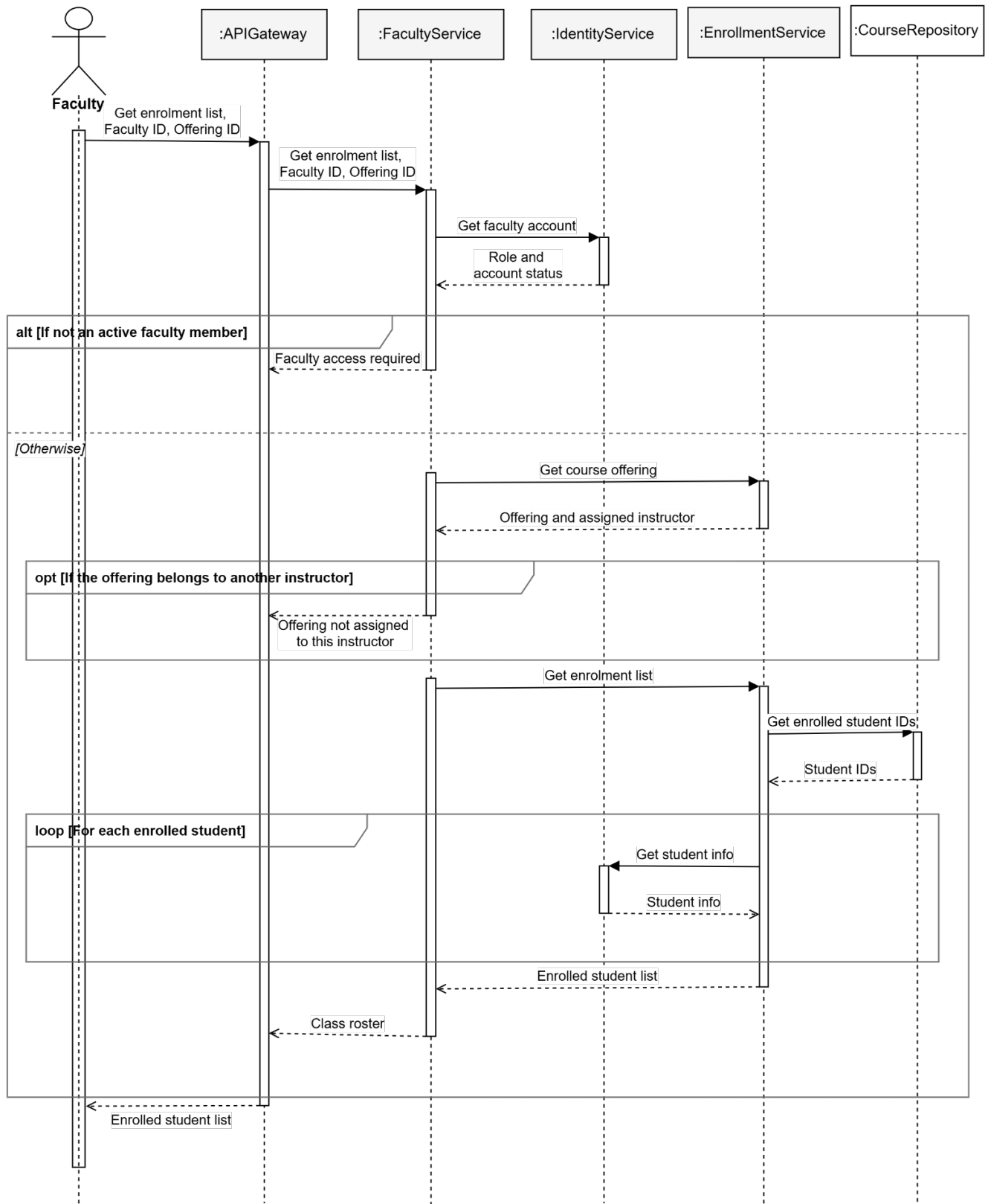
Sequence Diagram 4 - Personal Schedule Management (Student)



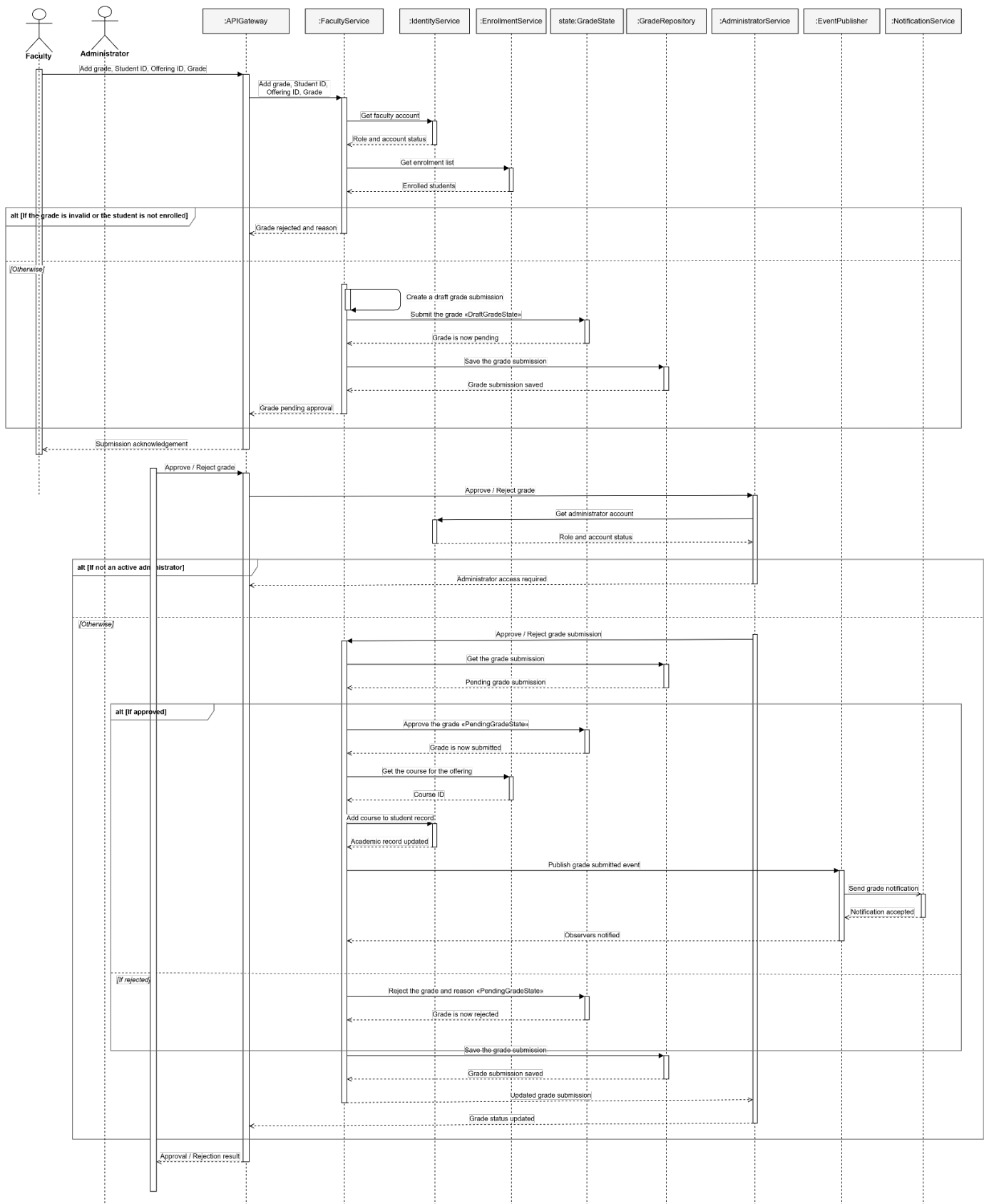
Sequence Diagram 5 - Academic Progress Tracking (Student)



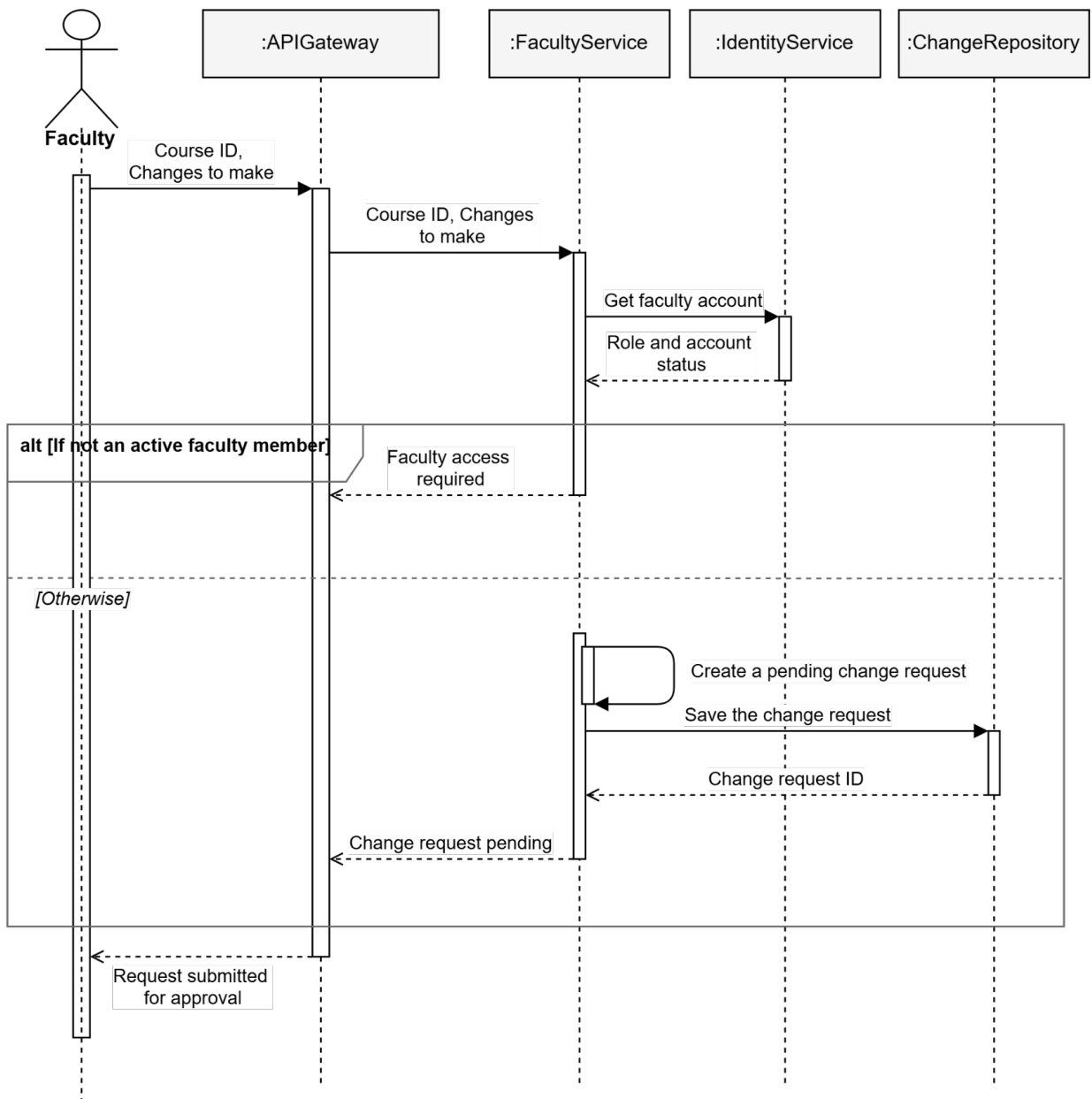
Sequence Diagram 6 - Class Roster Viewing (Faculty)



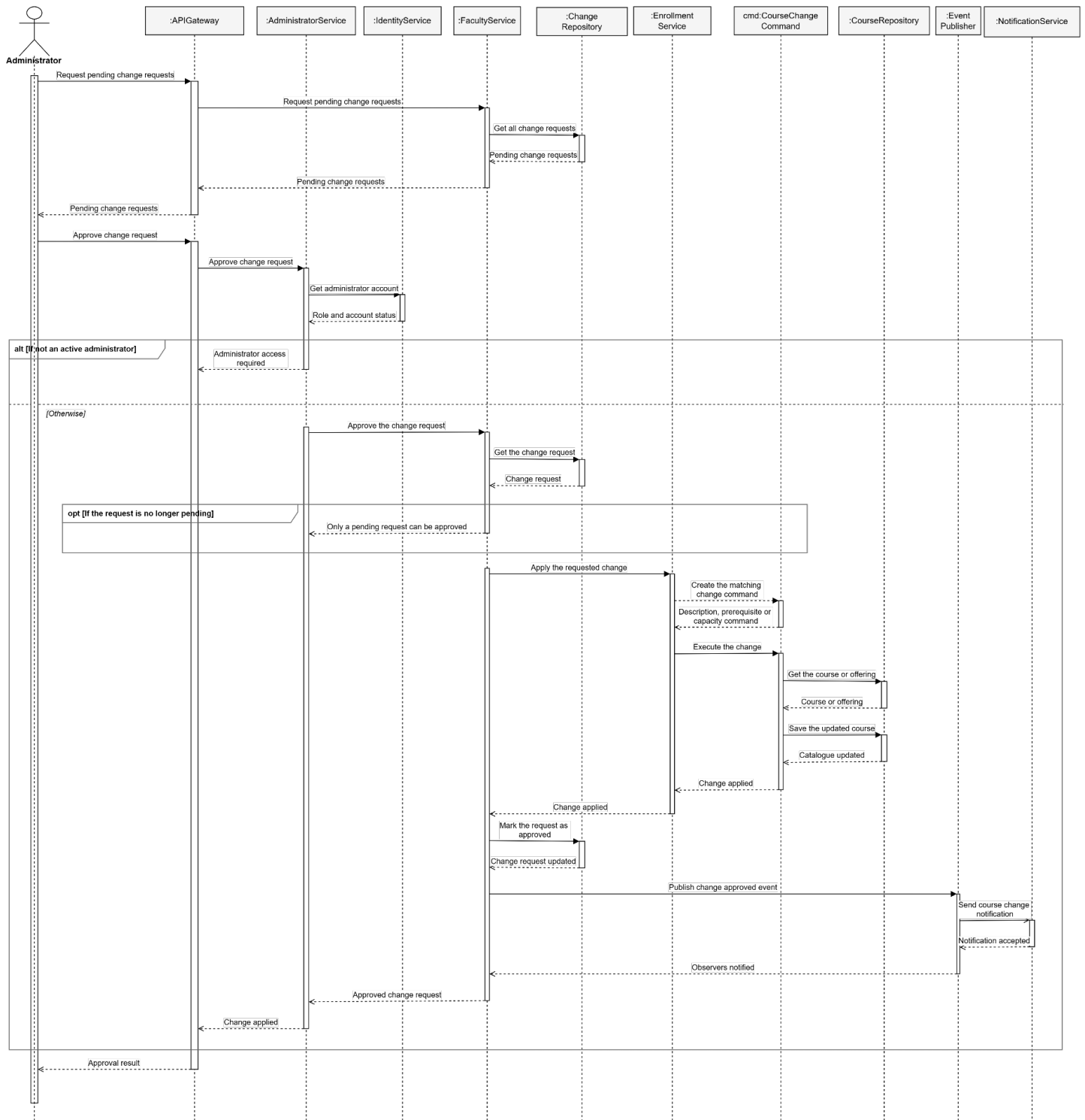
Sequence Diagram 7 - Grade Submission and Approval (Faculty + Administrator)



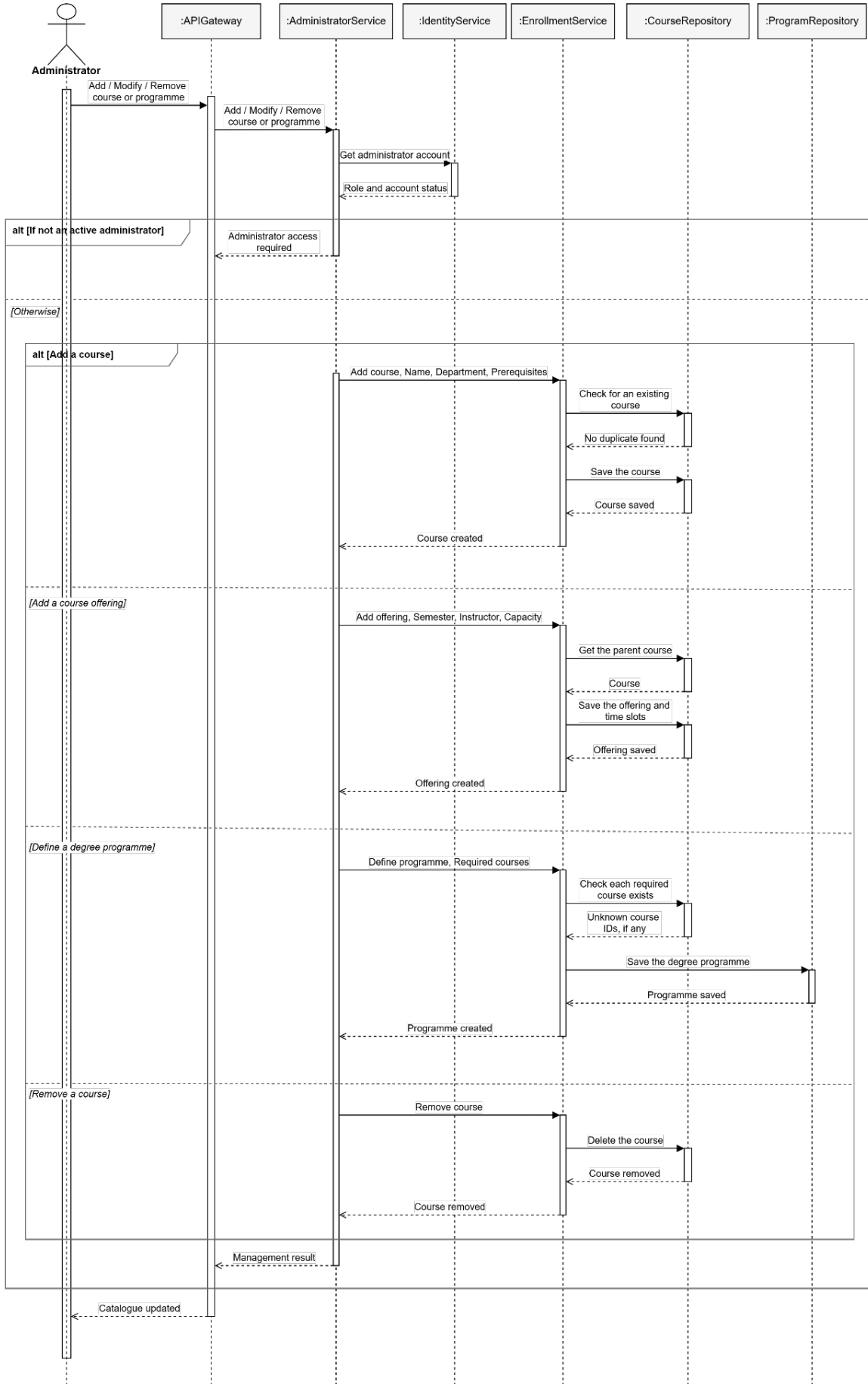
Sequence Diagram 8 - Course Change Request (Faculty)



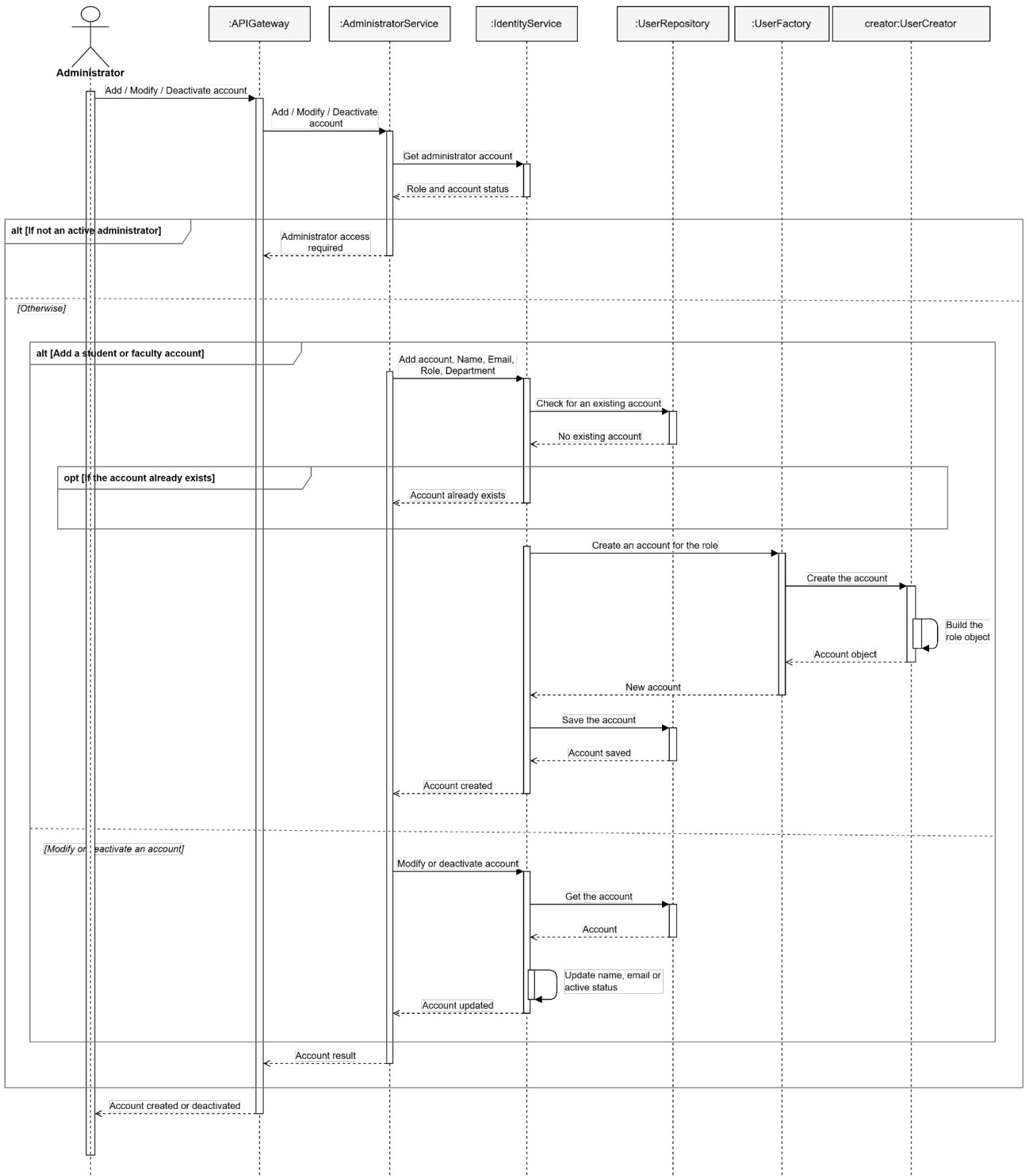
Sequence Diagram 9 - Course Change Approval (Administrator)



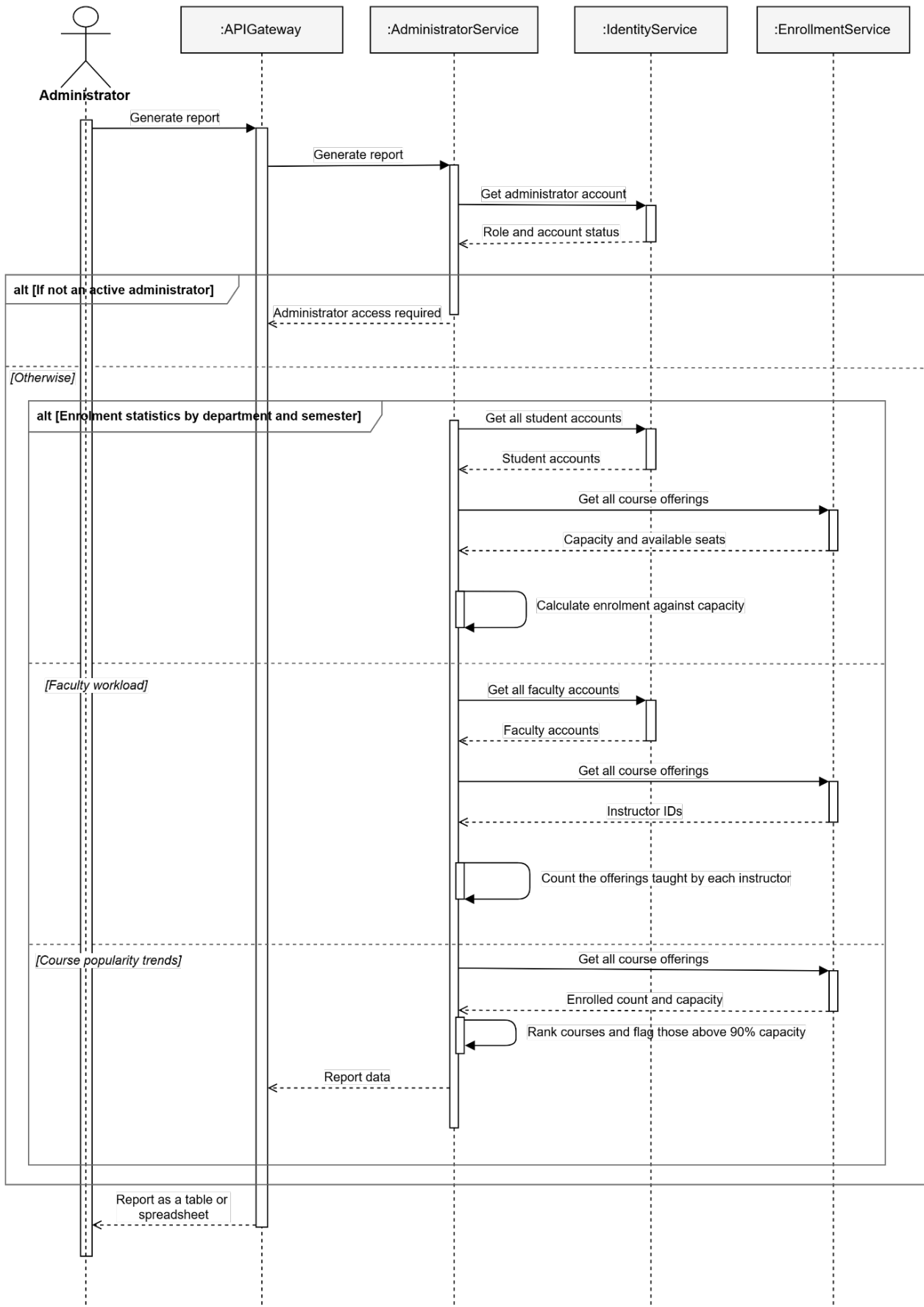
Sequence Diagram 10 - Course and Degree Programme Management (Administrator)



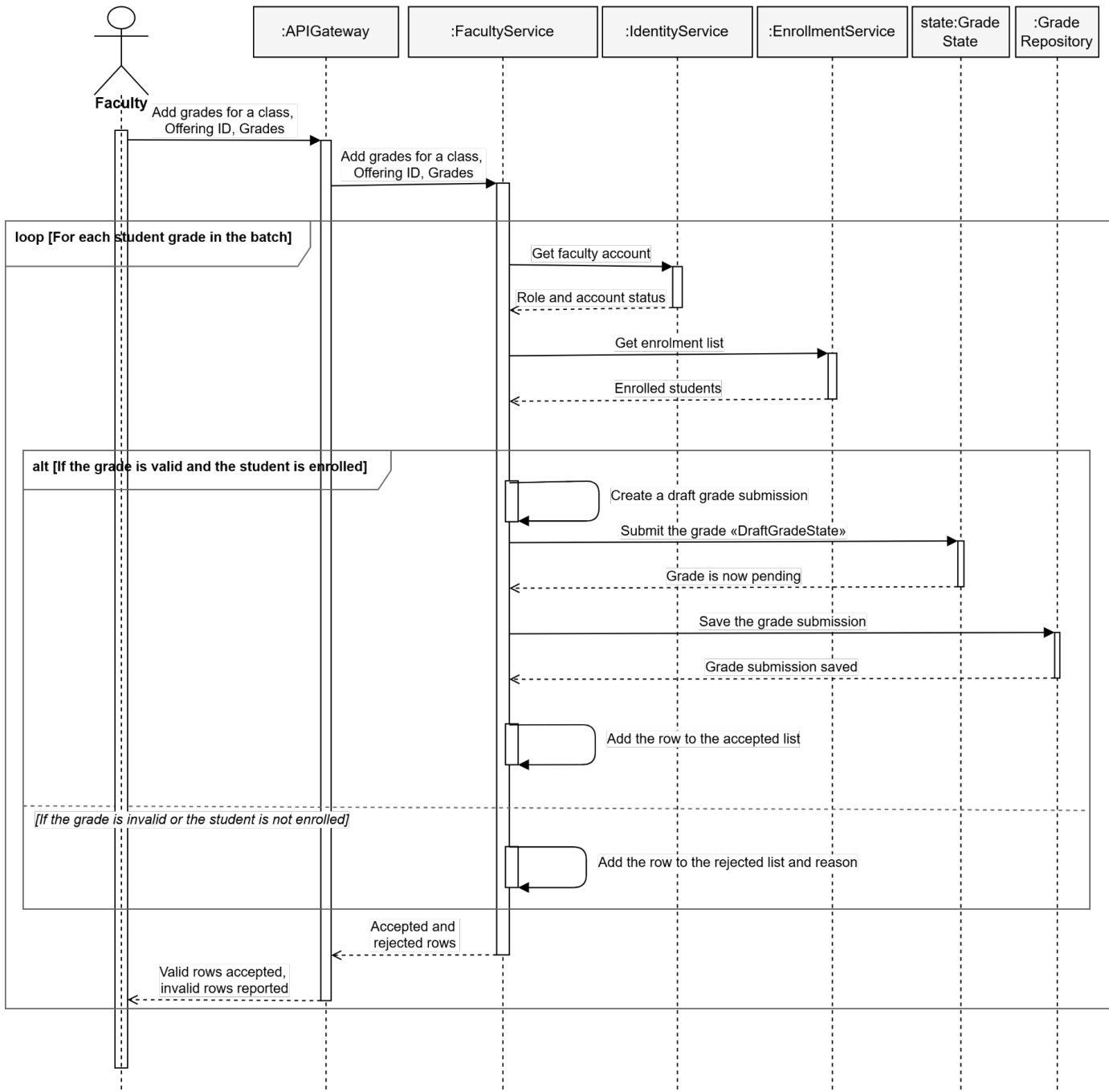
Sequence Diagram 11 - Student and Faculty Account Management (Administrator)



Sequence Diagram 12 - Reporting and Analytics (Administrator)



Sequence Diagram 13 - Batch Grade Submission with Partial Rejection (Faculty)



3.2. Activity Diagrams

Diagram 1 - Browse Course Catalogue

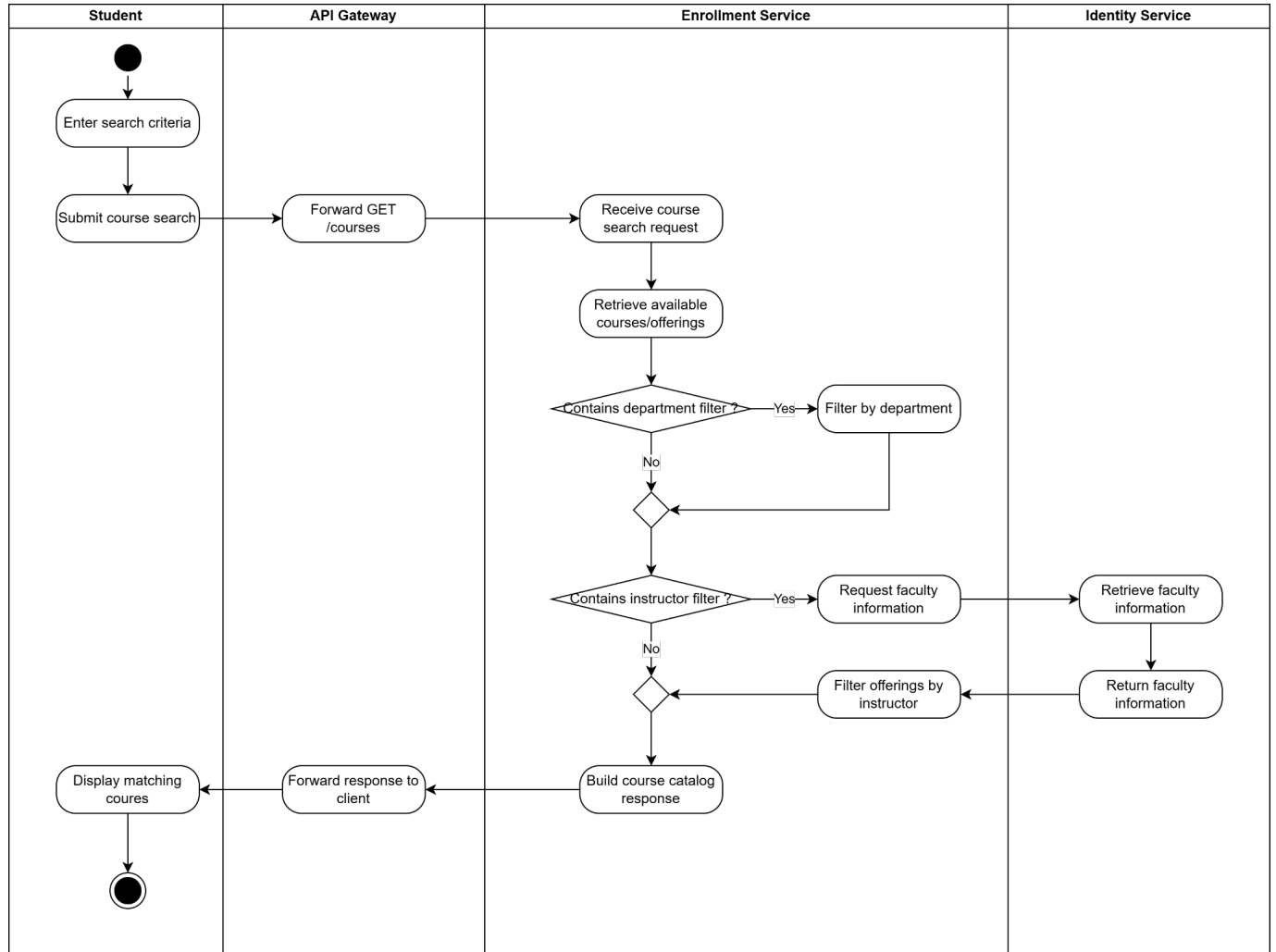


Diagram 2 - Student Registration

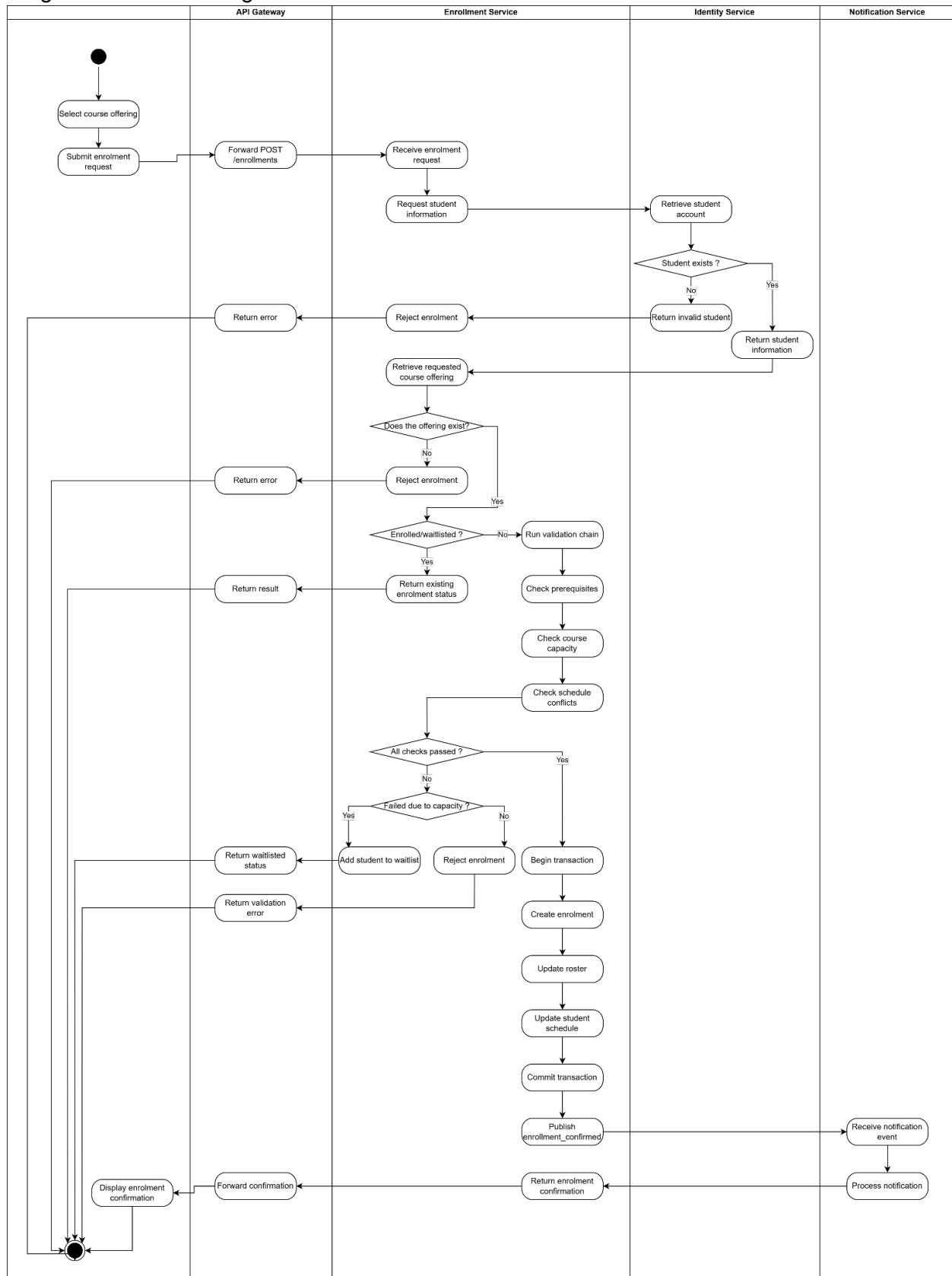


Diagram 3 - Drop Course

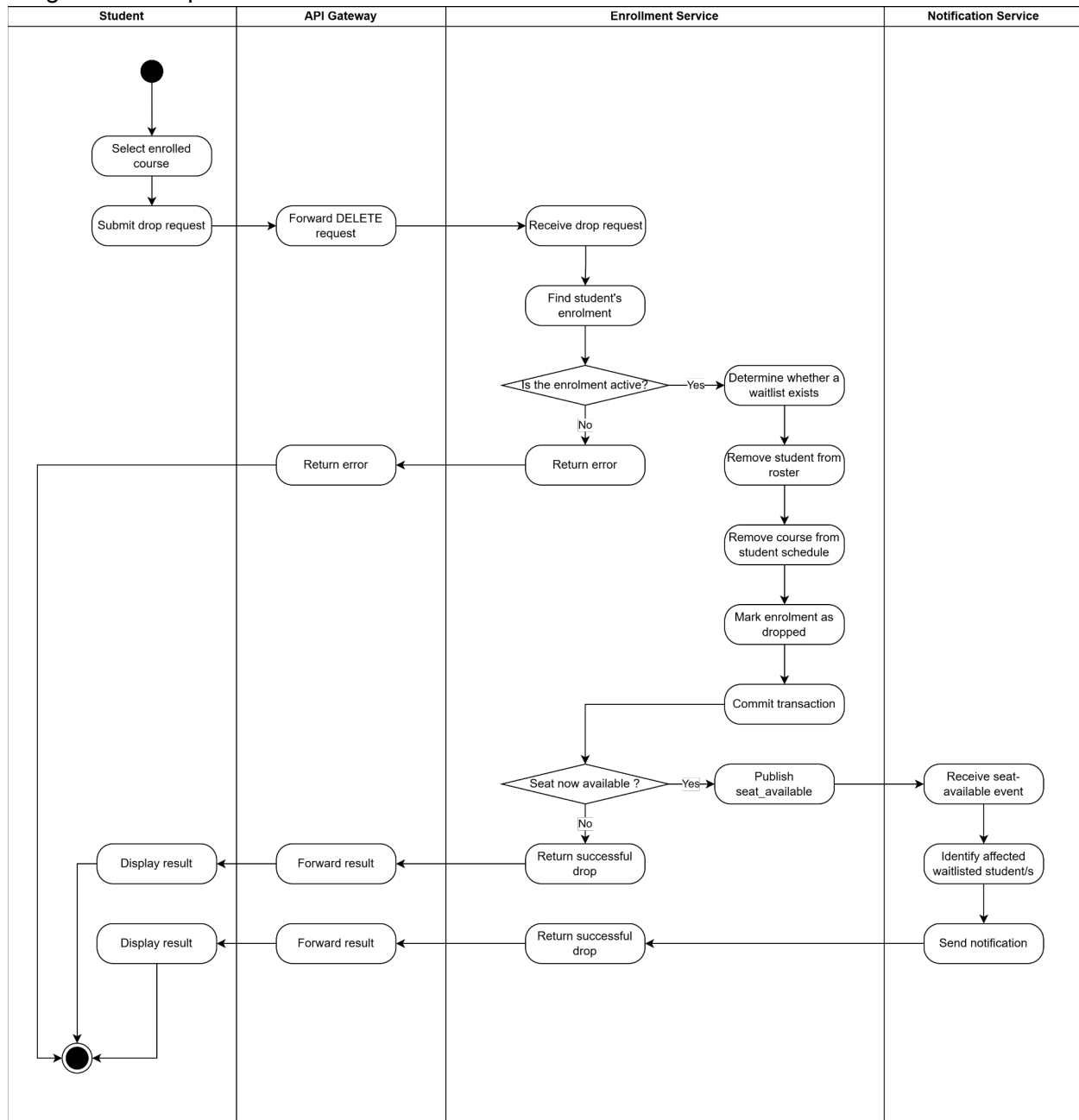


Diagram 4 - Personal Schedule Management

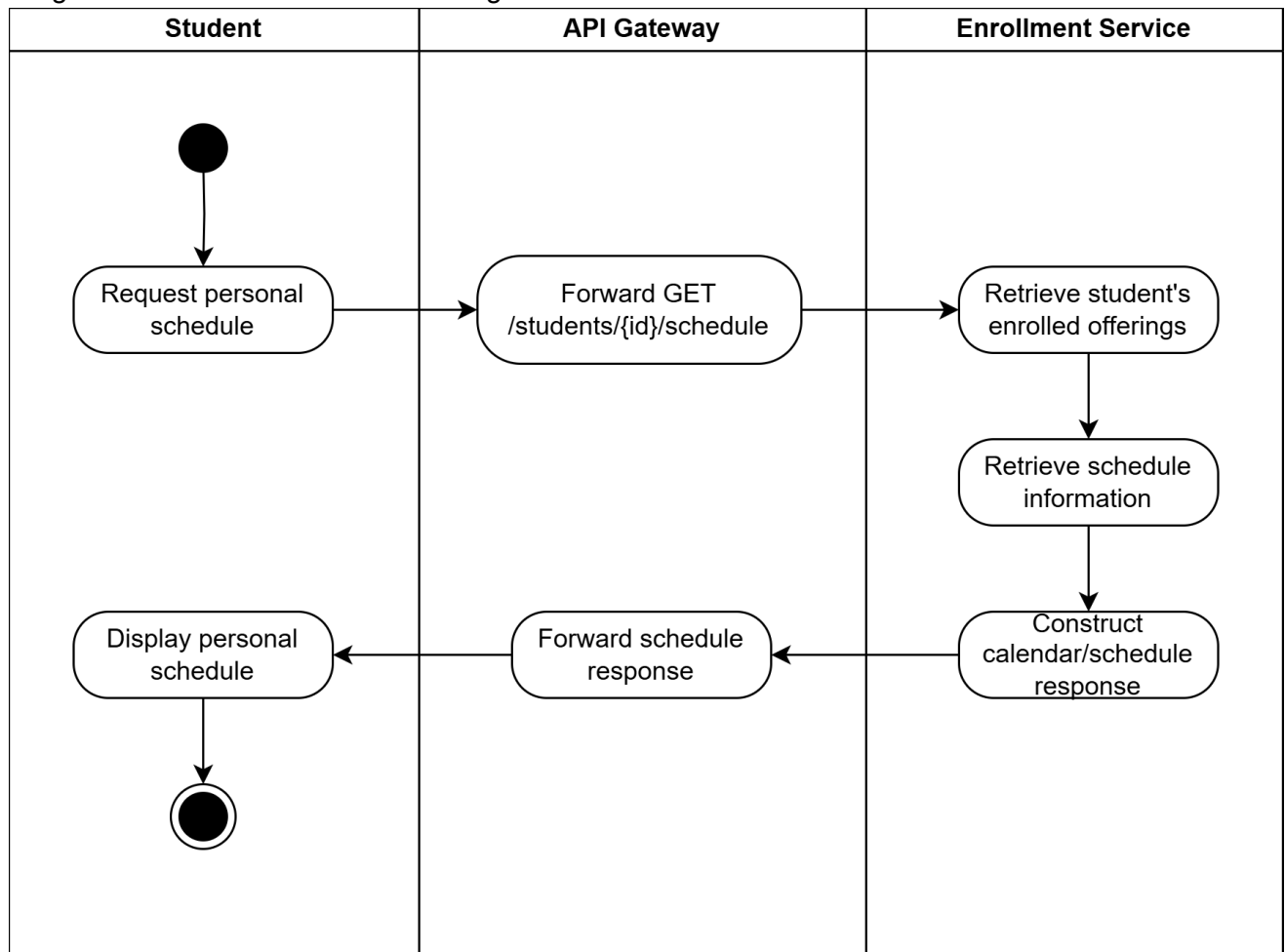


Diagram 5 - Academic Progress Tracking

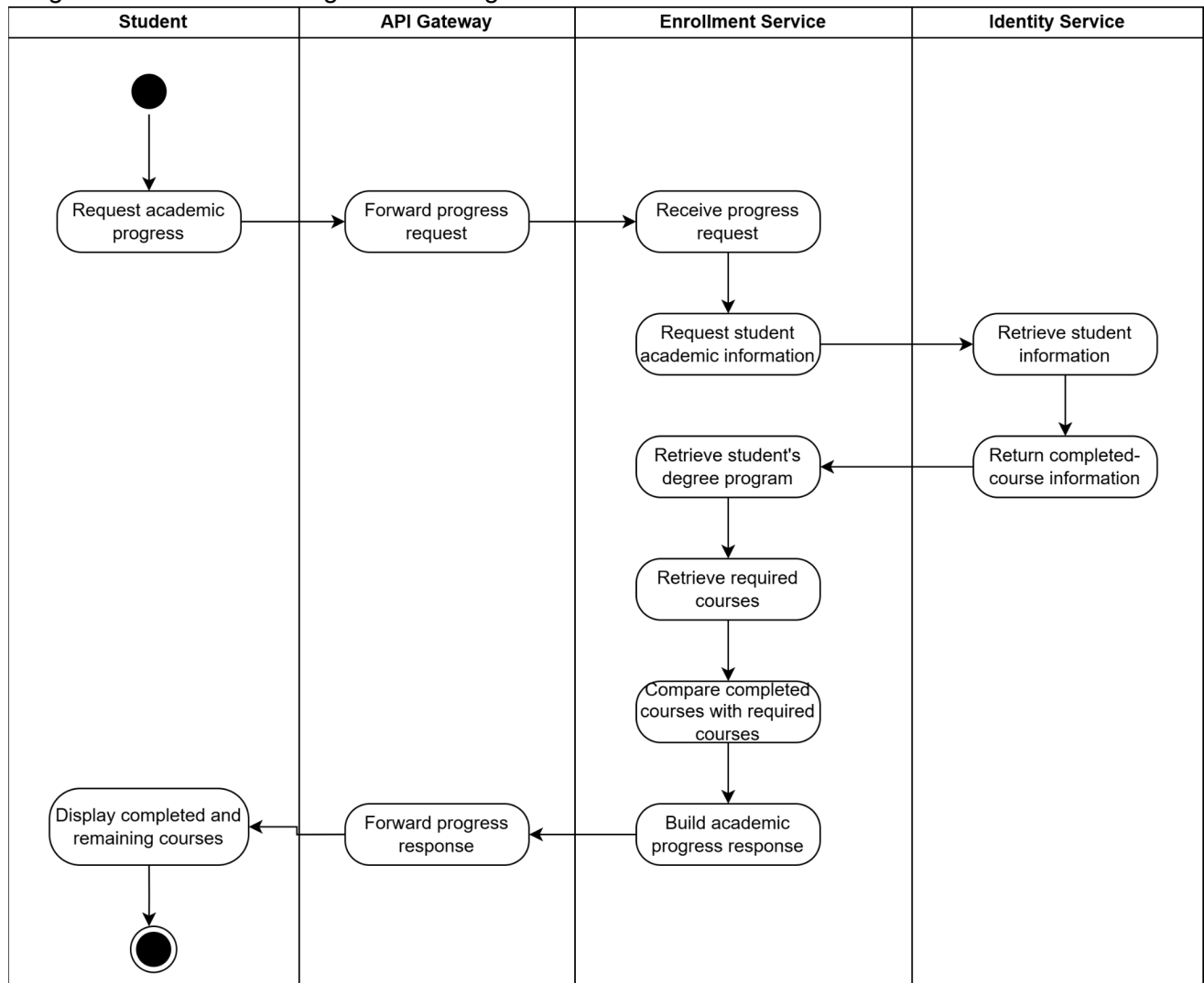


Diagram 6 - View Class Roster

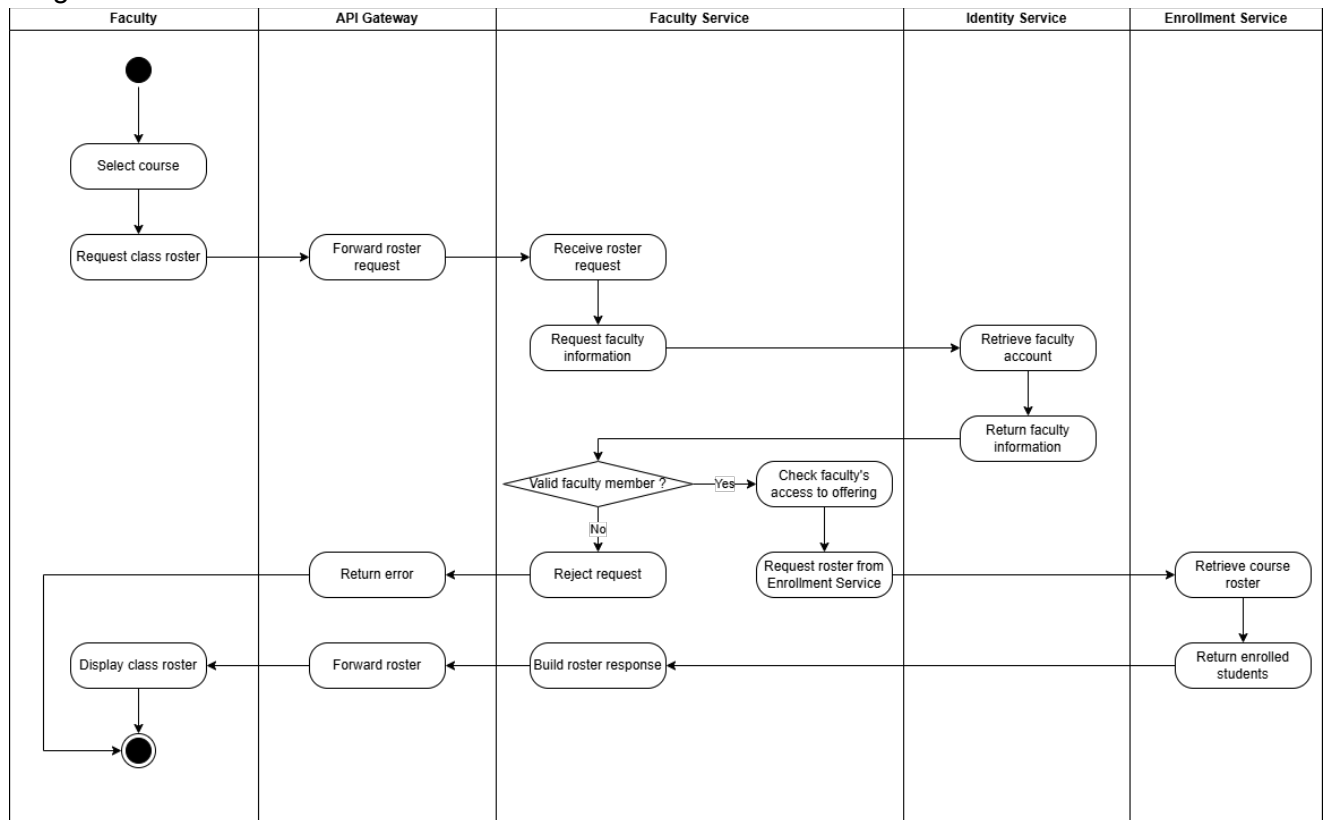


Diagram 7 - Grade Submission and Approval

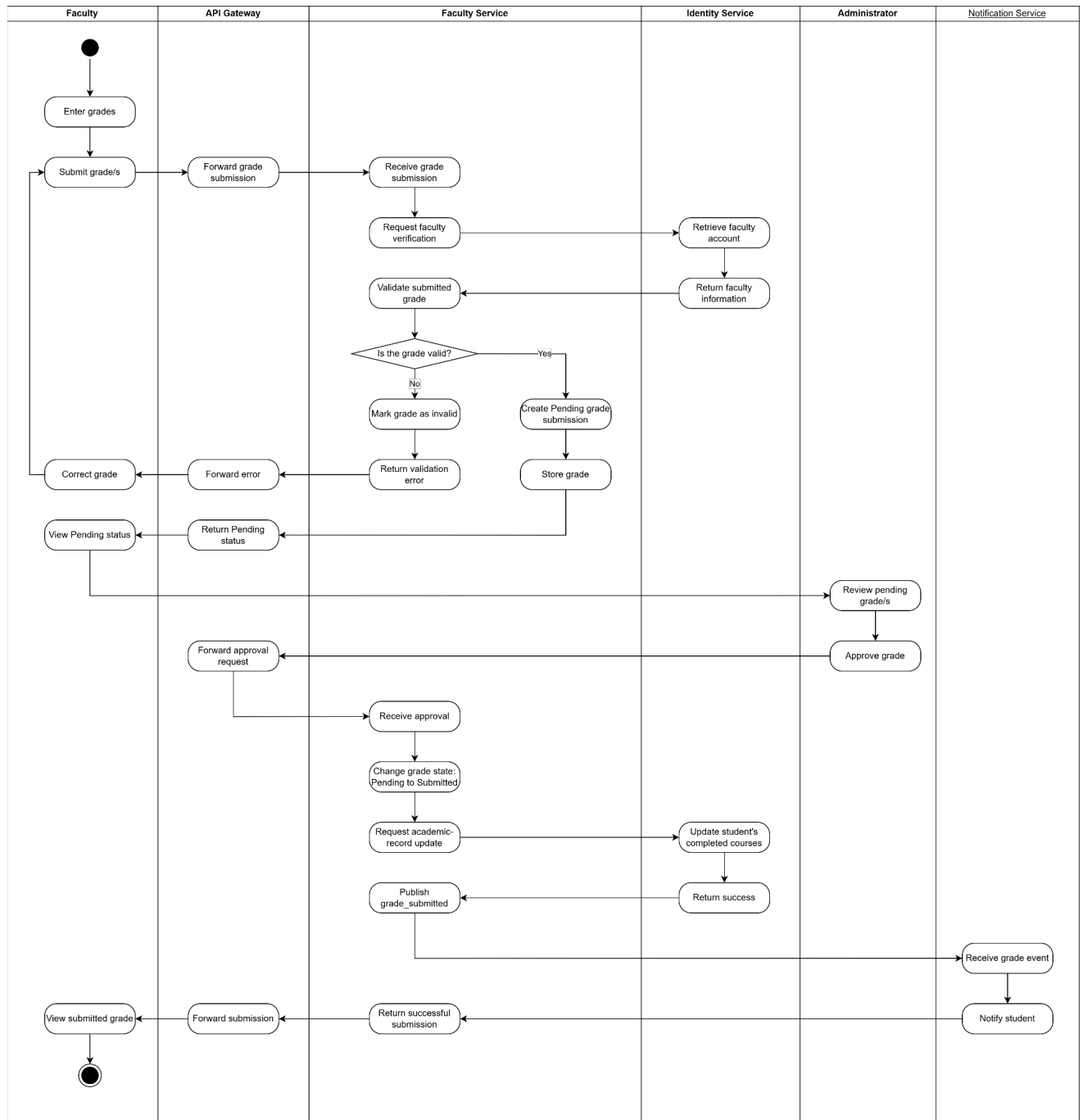


Diagram 8 - Course Change Request

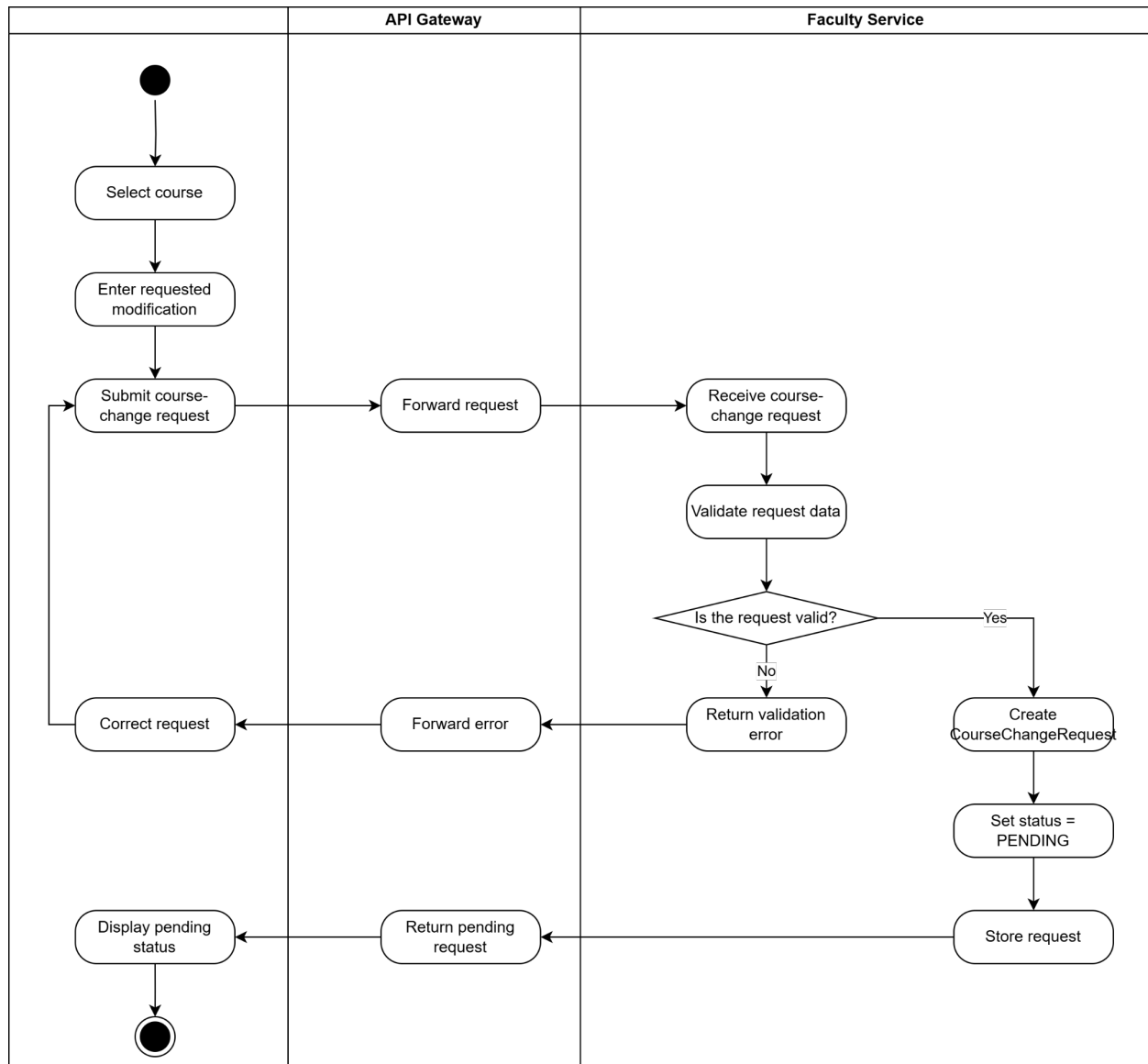


Diagram 9 - Course Change Approval

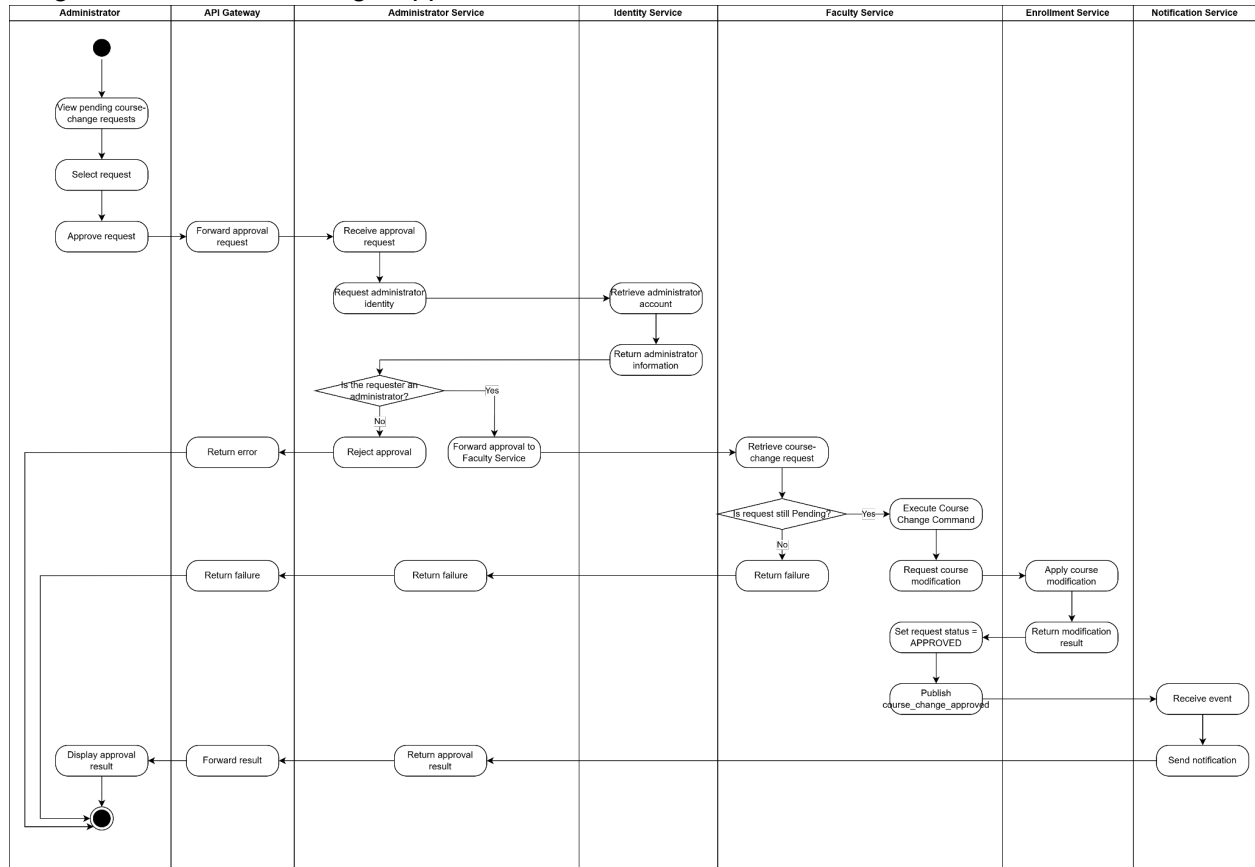


Diagram 10 - Course and Program Management

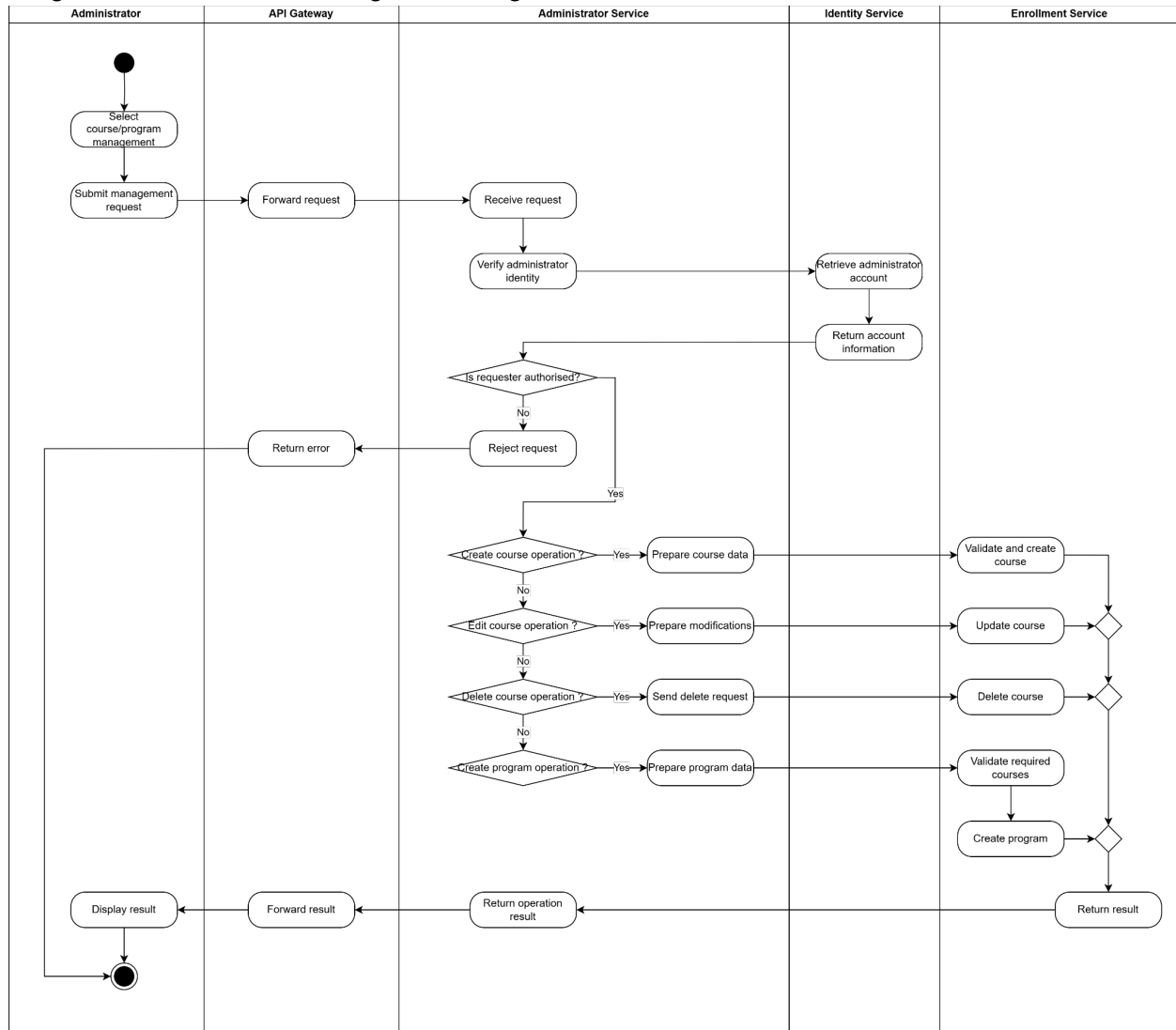


Diagram 11 - Student and Faculty Account Management

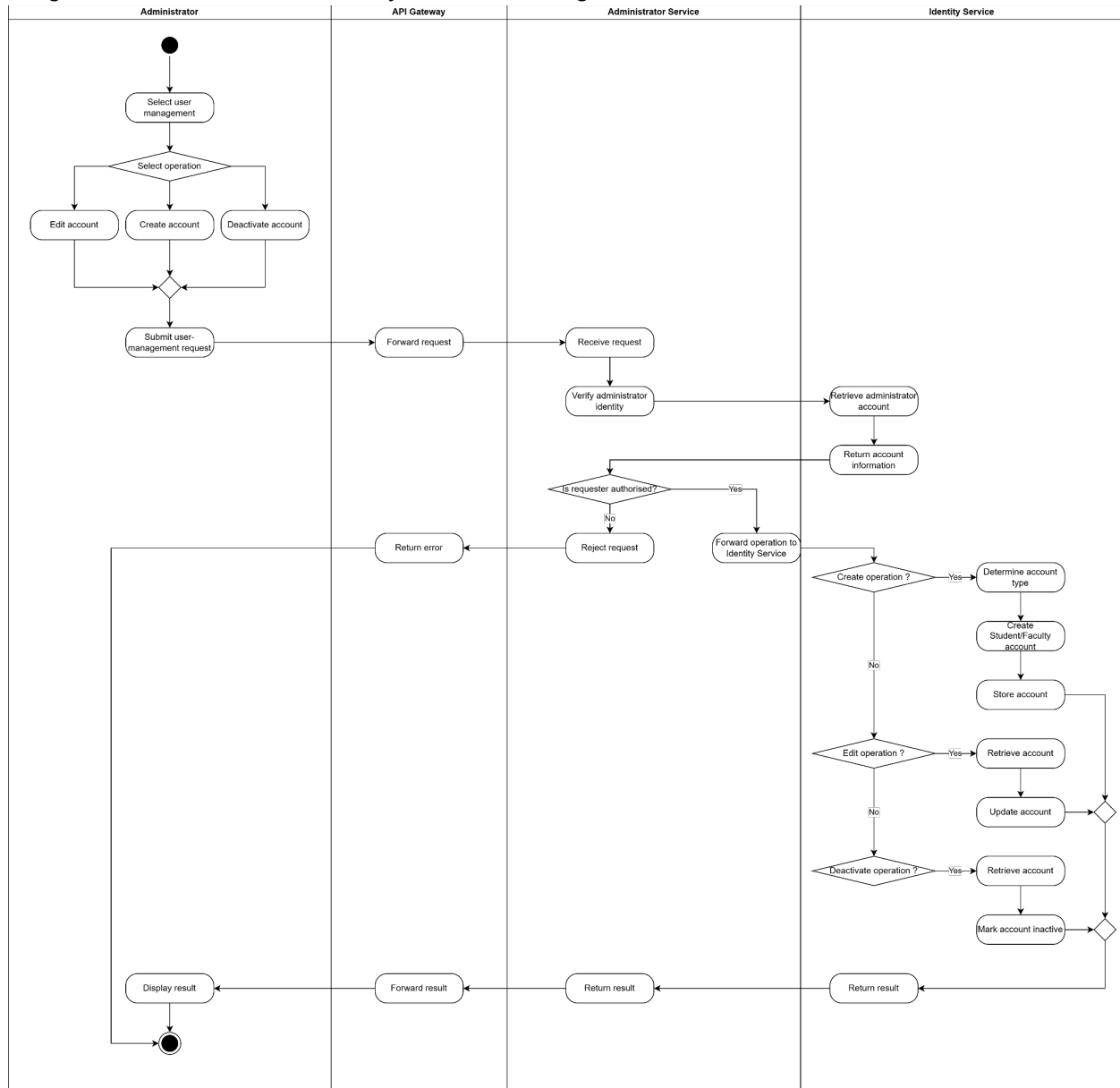
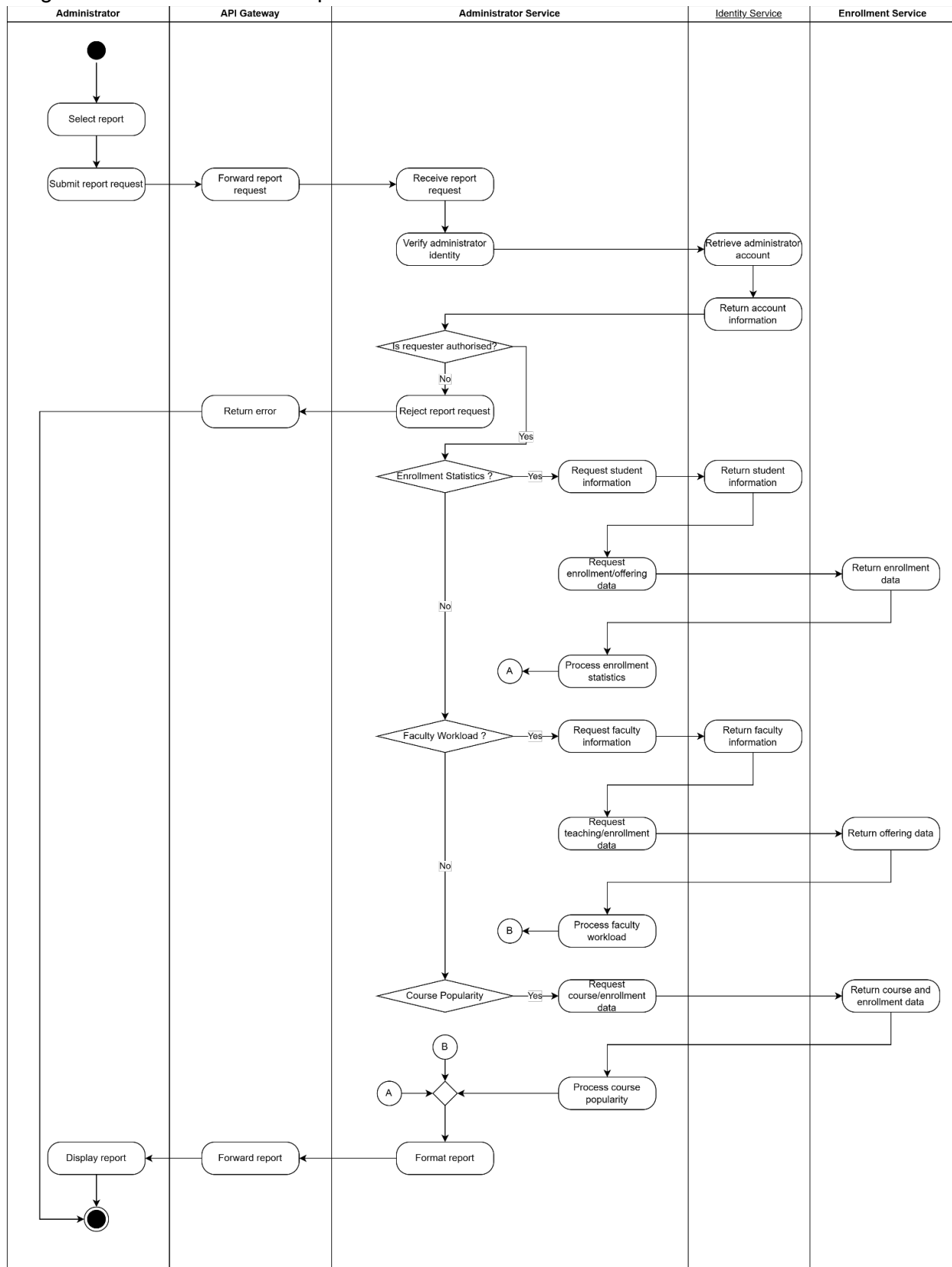
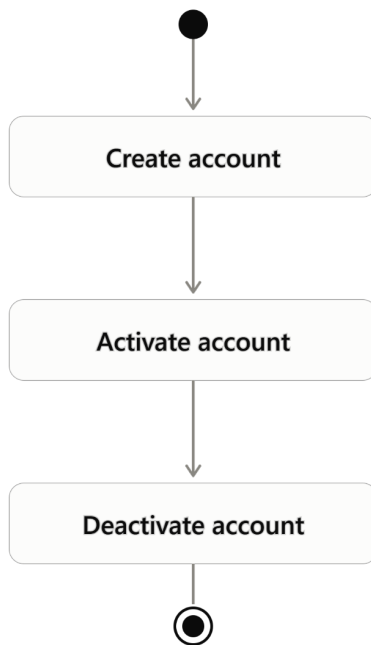


Diagram 12 - Administrative Report Generation

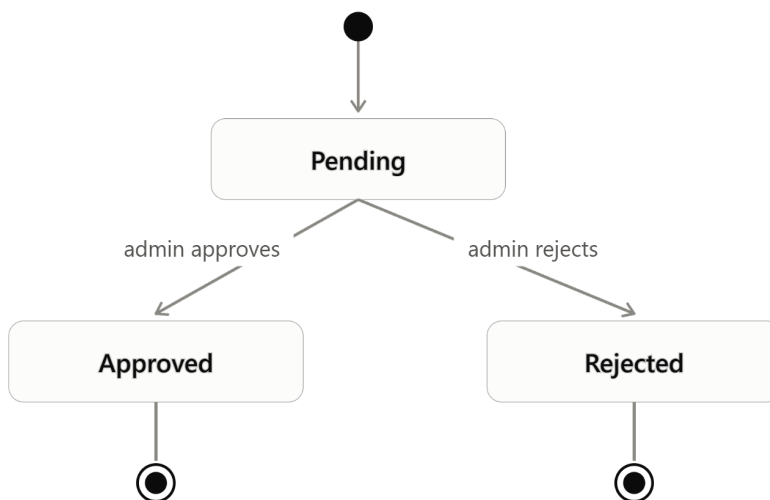


3.3. State Diagrams

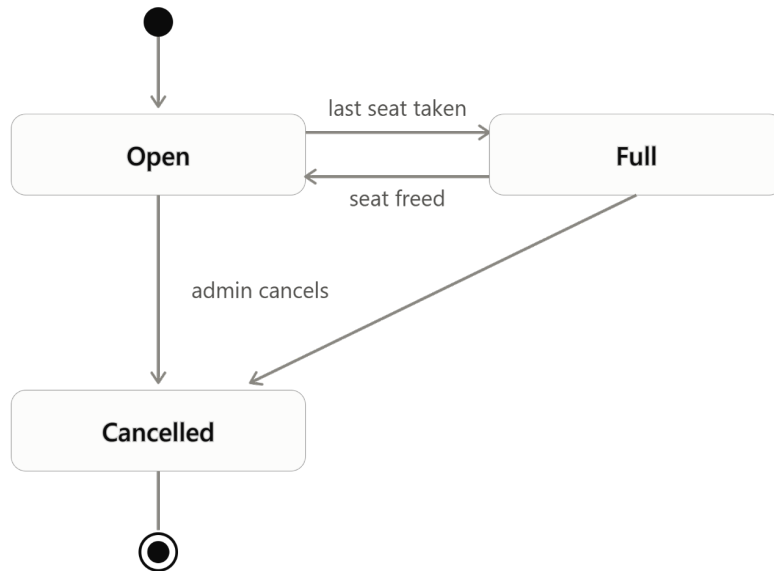
Account State Diagram



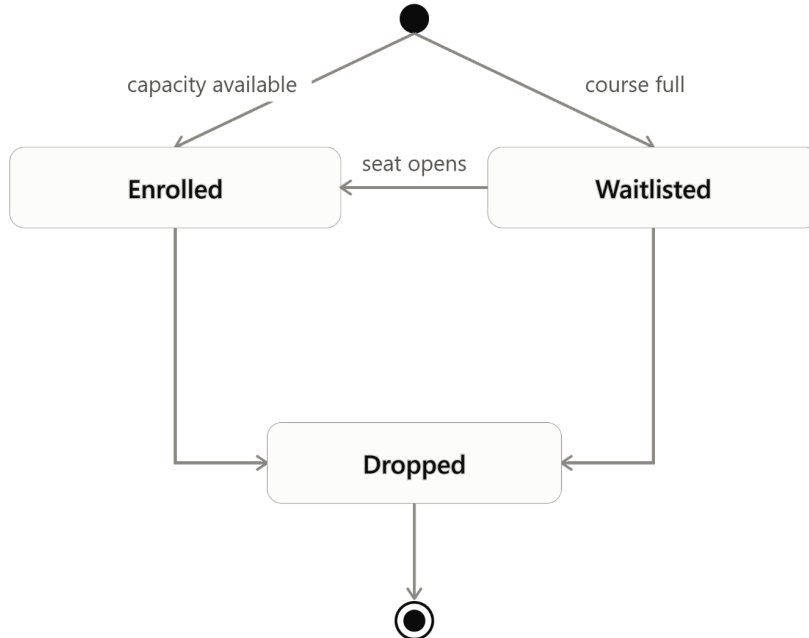
Course Change Request State Diagram



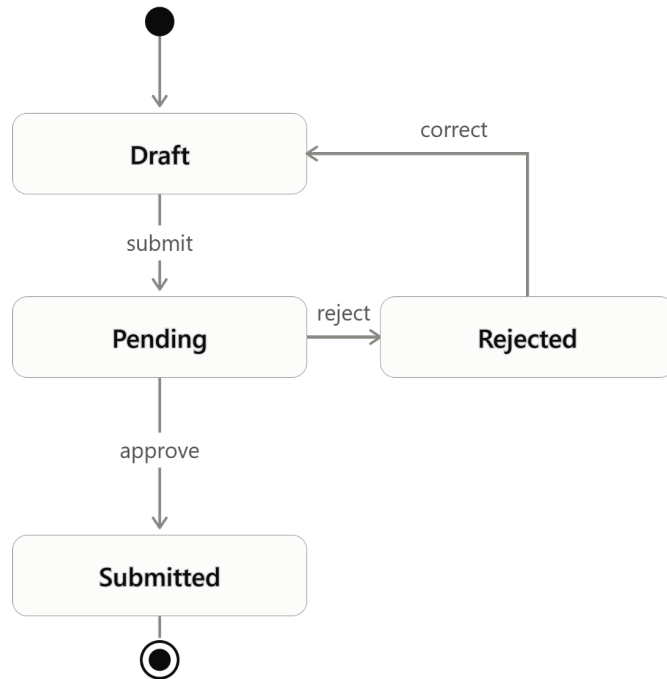
Course Offering State Diagram



Enrollment State Diagram



Grade State Diagram



4. Software Engineering Principles Demonstrated

4.1. Encapsulation

Encapsulation is applied by keeping an object's internal state behind the object that manages it. In Python, the code uses the conventional `_` prefix for internal attributes as access modifiers are not available in Python.

```
class EnrollmentValidator(ABC):
    def __init__(self):
        self._next_validator = None

    def set_next(self, validator):
        self._next_validator = validator
        return validator

    def validate(
        self, student: Student, offering: CourseOffering
    ) -> ValidationResult:
        result = self.check(student, offering)
        if not result.passed:
            return result

        if self._next_validator:
            return self._next_validator.validate(student, offering)
        return result

    @abstractmethod
    def check(self, student: Student, offering: CourseOffering) ->
ValidationResult:
    ...
```

The `_next_validator` attribute is treated as an internal implementation detail. Other parts of the system interact with the validator through `set_next()` and `validate()` rather than directly manipulating the chain's internal state. This keeps the chain's structure under the control of `EnrollmentValidator` and makes its internal representation less exposed to the rest of the application.

A similar approach is used by `UserFactory` with `_creators` and by `EventPublisher` with `_observers`.

4.2. Programming to an Interface

Programming to an interface means that code should depend on an abstraction rather than a specific implementation wherever possible. The codebase uses Python's abstract base classes to define contracts for the design-pattern implementations as interfaces are not available in Python.

```
from abc import ABC, abstractmethod

class CourseChangeCommand(ABC):
    @abstractmethod
    def execute(self, courses):
        ...

class ChangeDescriptionCommand(CourseChangeCommand):
    def __init__(self, course_code, new_description):
        self.course_code = course_code
        self.new_description = new_description

    def execute(self, courses):
        course = courses.get_course(self.course_code)
        if course is None:
            raise ValueError("Course not found.")

        course.description = self.new_description
        courses.save_course(course)
```

Code that works with a `CourseChangeCommand` does not need to know which concrete command it has received. The same principle is used by `UserCreator`, `Observer`, `GradeState`, and `EnrollmentValidator`.

This makes the design easier to extend because a new implementation can be introduced while existing code continues to depend on the same abstraction.

4.3. Composition Over Inheritance

The codebase uses inheritance where it represents a genuine **is-a** relationship, particularly for the pattern abstractions, but relies on composition to assemble behaviour wherever appropriate.

```
def build_course_change_command(action, payload):
    if action == "change_description":
        return ChangeDescriptionCommand(
            payload["course_code"], payload["new_description"]
        )
    if action == "add_prerequisite":
        return AddPrerequisiteCommand(
            payload["course_code"], payload["prerequisite_code"]
        )
    if action == "change_capacity":
        return ChangeCapacityCommand(payload["offering_id"],
payload["new_capacity"])
    raise ValueError("Unsupported course-change command.")
```

`PrerequisiteValidator`, `TimeConflictValidator`, and `CapacityValidator` do not inherit from one another. Instead, they are composed into a chain by giving each validator a reference to the next validator.

This is preferable to creating a large inheritance hierarchy because each validation rule remains an independent component that can be rearranged, replaced, or extended without changing the other validators. Inheritance is retained only where the abstraction itself is meaningful, such as `PrerequisiteValidator` being an `EnrollmentValidator`.

Another clear example is `EventPublisher`, which contains a collection of observers rather than inheriting from each observer type.

4.4. SOLID Principles

4.4.1. Single Responsibility Principle

The Single Responsibility Principle states that a class should have one primary reason to change.

```

class CapacityValidator(EnrollmentValidator):
    def check(self, student: Student, offering: CourseOffering) ->
    ValidationResult:
        if offering.is_full:
            return ValidationResult(
                False,
                ValidationFailure.CAPACITY,
                "Course offering is full.",
            )
        return ValidationResult(True)

```

CapacityValidator is responsible only for checking whether an offering has capacity. It does not handle prerequisites, timetable conflicts, notifications, or enrolment persistence. The same separation exists between PrerequisiteValidator and TimeConflictValidator, and between services such as the Faculty Service and Notification Service. This keeps individual components focused and reduces the impact of changes.

4.4.2. Open/Closed Principle

The Open/Closed Principle states that software should be open for extension but closed for unnecessary modification.

For example a new validation rule like the code snippet given below could be easily added to the validation chain **(not present in the codebase)**

```

class CreditLimitValidator(EnrollmentValidator):
    def check(self, student, offering):
        if self._exceeds_credit_limit(student, offering):
            return ValidationResult(
                False,
                ValidationFailure.CREDIT_LIMIT,
                "Credit limit exceeded.",
            )

        return ValidationResult(True)

```

A new validation rule can be implemented as another `EnrollmentValidator` without modifying the existing prerequisite, time-conflict, or capacity validators. The same idea applies to the design-pattern implementations: a new `CourseChangeCommand` can implement `CourseChangeCommand` without changing the existing command classes

4.4.3. Liskov Substitution Principle

The Liskov Substitution Principle requires implementations of an abstraction to be usable wherever that abstraction is expected without breaking the expected behaviour.

```
class UserFactory:
    """Creates user types without coupling identity logic to API
    routing."""

    def __init__(self):
        self._creators = {
            UserRole.STUDENT: StudentCreator(),
            UserRole.FACULTY: FacultyCreator(),
            UserRole.ADMINISTRATOR: AdministratorCreator(),
        }

    def create(
        self,
        role: UserRole,
        user_id: str,
        name: str,
        email: str,
        **kwargs,
    ) -> User:
        return self._creators[role].create(user_id, name, email, **kwargs)
```

Each concrete creator implements the same `create()` contract:

```
class StudentCreator(UserCreator):
    def create(self, user_id: str, name: str, email: str, **kwargs) ->
    User:
        return Student(user_id, name, email, UserRole.STUDENT)

class FacultyCreator(UserCreator):
    def create(self, user_id: str, name: str, email: str, **kwargs) ->
    User:
        department = kwargs.get("department", "")
        return Faculty(user_id, name, email, UserRole.FACULTY, department)

class AdministratorCreator(UserCreator):
    def create(self, user_id: str, name: str, email: str, **kwargs) ->
    User:
        return Administrator(user_id, name, email, UserRole.ADMINISTRATOR)
```

`UserFactory` does not need to know which concrete creator it is calling; every creator can substitute for `UserCreator` because each provides the required `create()` operation and returns a `User`. This is a genuine example of substitutability in the current implementation.

4.4.4. Interface Segregation Principle

The Interface Segregation Principle states that a class should not be forced to depend on methods that it does not need.

```
class Observer(ABC):
    @abstractmethod
    def update(self, event: DomainEvent) -> None:
        ...
```

The interface contains only the operation required by the event publisher: `update()`.

The concrete notification observer implements precisely that contract:

```
class HttpNotificationObserver(Observer):
    def __init__(self, http_client, notification_service_url: str):
        self.http_client = http_client
        self.notification_service_url = notification_service_url

    def update(self, event: DomainEvent) -> None:
        self.http_client.post(
            f"{self.notification_service_url}/events",
            json={"name": event.name, "payload": event.payload},
            timeout=2,
        )
```

EventPublisher therefore depends only on the small Observer contract:

```
class EventPublisher:
    """Publishes domain events to subscribed observers."""

    def __init__(self):
        self._observers = []

    def subscribe(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def publish(self, event):
        for observer in tuple(self._observers):
            try:
                observer.update(event)
            except Exception:
                # Notification failure must not roll back the business
                transaction.
            pass
```

4.4.5. Dependency Inversion Principle

The Dependency Inversion Principle states that higher-level components should depend on abstractions rather than directly on concrete implementations.

EventPublisher does not depend on HttpNotificationObserver:

```
class EventPublisher:
    """Publishes domain events to subscribed observers."""

    def __init__(self):
        self._observers = []

    def subscribe(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def publish(self, event):
        for observer in tuple(self._observers):
            try:
                observer.update(event)
            except Exception:
                # Notification failure must not roll back the business
                transaction.
            pass
```

It depends only on the Observer abstraction. The concrete implementation is supplied externally when the service is configured:

```
event_publisher = EventPublisher()
unit_of_work = UnitOfWork(
    [course_repository, enrollment_repository, schedule_repository]
)
http_client = httpx.Client()
event_publisher.subscribe(
    HttpNotificationObserver(http_client, NOTIFICATION_SERVICE_URL)
)
```

The publisher therefore does not need to know how notifications are delivered. It only knows that its dependency provides `update()`.

This means the event-publishing logic can remain unchanged even if the concrete observer implementation is replaced.

5. Conclusion

NexusEnroll has been designed using a microservices architecture that separates the system into an API Gateway, Identity Service, Enrollment Service, Faculty Service, Administrator Service, and Notification Service. Each service is responsible for a specific business area and communicates with other services through REST APIs. This separation reduces unnecessary coupling and makes the overall system easier to understand, maintain, and extend.

The API Gateway acts as the common entry point for the Web UI and Mobile App and routes requests to the appropriate backend service. The Identity Service manages students, faculty members, administrators, roles, and completed course information. The Enrollment Service handles courses, course offerings, programmes, enrolment, waitlists, schedules, academic progress, and validation of prerequisites, timetable conflicts, and capacity. The Faculty Service manages class rosters, grade submissions, grade state transitions, and course change requests, while the Administrator Service handles privileged operations such as user management, enrollment overrides, approvals, programme and course management, and reporting. The Notification Service is kept separate from core business operations and receives domain events generated by the Enrollment and Faculty Services.

The implementation also applies five object-oriented design patterns: **Factory Method** for user creation, **Chain of Responsibility** for enrolment validation, **Observer** for notifications, **Command** for course change operations, and **State** for managing the grade submission lifecycle. These patterns improve modularity, code reuse, and extensibility while keeping complex workflows easier to manage.

The selected architecture provides several important benefits. Maintainability is improved because changes can be made within a specific service without significantly affecting unrelated functionality. Scalability is also improved because services can be scaled according to their own workloads, such as increasing resources for the Enrollment Service during peak registration periods. The API Gateway provides a consistent access point for web and mobile clients while also controlling service exposure. Sensitive Identity write operations are performed internally through the Administrator Service, while read access can be provided through the Gateway.

Overall, the combination of microservices and object-oriented design patterns gives NexusEnroll a **modular, maintainable, scalable, and extensible architecture** that supports current university requirements while also providing a strong foundation for future expansion.